

Building **Production-Ready Workloads** for Microsoft Fabric

From the ISV business case and architecture to secure delivery, operations, and distribution.

01

Understand

02

Develop

03

Production

04

Distribute

Contents

P Choose your path **0** Before you begin

First run

Common pitfalls

Understand

1 What a workload is, and why an ISV would build one

1.1 The ISV business case: bring your service to Fabric data

1.2 What Fabric gives you, and what a workload adds

1.3 The toolkit, when to use it, and its limits

2 How a workload runs: architecture, the host, and one request

2.1 The Fabric host, your frontend, and an optional backend

2.2 Items as native artifacts, with explicit integrations

2.3 A request, step by step

3 The manifest package: the contract with Fabric

3.1 Workload, product, and paired item manifests

3.2 Identity and naming

4 Identity and access with Microsoft Entra

4.1 Delegated frontend tokens and backend OBO

4.2 Entra applications, scopes, and service identities

Develop

5 The toolkit and the development environment

- 5.1 The starter kit, setup script, and Entra applications
- 5.2 Dev Server, Dev Gateway, and the Hello World checkpoint

6 Building an item: editor, data, and capabilities

- 6.1 The item, its editor, and how it surfaces
- 6.2 Separating item definitions from OneLake data
- 6.3 Remote jobs, lifecycle events, and other capabilities

7 Developing with AI assistance

- 7.1 Repository guidance: useful, mutable, and versioned
- 7.2 The Copilot agent and the optional community UX MCP
- 7.3 What it generates, and keeping it honest

8 Diagnostics and debugging

- 8.1 Reading the chain, and the token and manifest boundaries
- 8.2 The Dev Gateway, the iframe boundary, and correlation

9 Illustration: GreenGrid

- 9.1 What GreenGrid is
- 9.2 The scoring service, in Python
- 9.3 Milestone 1: the workload calls the service
- 9.4 Milestones 2 and 3: real data and a native screen

Go to production

10 From developer mode to production

- 10.1 What changes
- 10.2 What to verify across the transition

11 Hosting, domain, and identity

- 11.1 Hosting, the verified domain, and the resource ID
- 11.2 User delegation, backend credentials, and managed identities

12 Security and compliance

- 12.1 Data boundaries, labels, and publisher responsibility
- 12.2 Secrets, telemetry, monitoring, and support

13 Packaging, validation, and CI/CD

- 13.1 Package build, schema checks, and publishing validation
- 13.2 Automating the pipeline

14 Patterns and anti-patterns

- 14.1 Patterns that hold up
- 14.2 Anti-patterns to avoid

Distribute

15 Make it available in your tenant

- 15.1 Admin Portal upload, activation, and assignment
- 15.2 Internal publishing with Org.[Name]

16 Publish across tenants: Workload Hub and Microsoft Marketplace

- 16.1 Selected tenants, Preview, and GA
- 16.2 Publishing requirements, commerce, and support
- 16.3 Choosing a path

17 The post-publish lifecycle

- 17.1 Updates, migration, and deprecation
- 17.2 Monitoring, consent, rollback, and feature flags

18 Recap and next steps

Appendices

- [Appendix A: Manifest package reference](#)
- [Appendix B: Setup, development, package, and validator commands](#)
- [Appendix C: AI guidance reference](#)
- [Appendix D: Python service reference](#)
- [Appendix E: Release and compliance checklist, diagnostics quick reference](#)
- [Appendix F: Glossary and resources](#)
- [Appendix G: Quick reference](#)

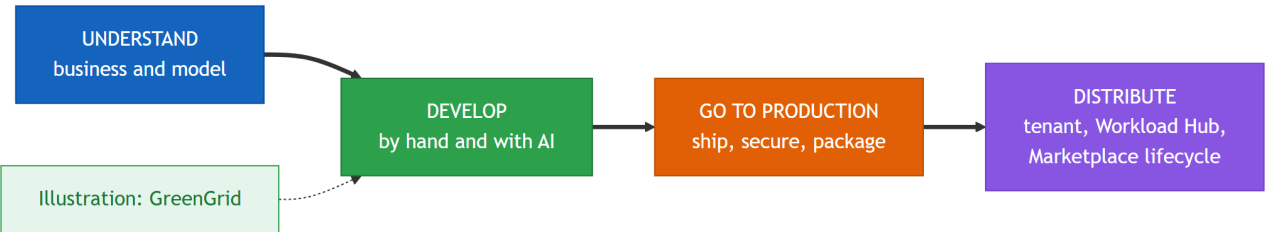
Microsoft Fabric ships with lakehouses, notebooks, pipelines, reports, and many other items. An independent software vendor (ISV) or software development company (SDC) may already have a useful SaaS, PaaS, API, or repeatable client solution, but no product surface where Fabric users work. A workload creates that surface while the service and intellectual property remain on infrastructure the publisher operates.

For the customer, this can shorten the path from governed data to a useful result. For the publisher, it can reduce repeated integration work, support recurring use inside Fabric, and create a route to customers whose data teams already use the platform. Those are product goals to validate, not outcomes Fabric guarantees.

This manuscript teaches the workload model in four movements. First, why an ISV would build one and how Fabric runs it. Second, how to develop an item with the toolkit, including the local loop, definitions, OneLake data, AI guidance, and diagnostics. Third, how to host, secure, package, and operate it. Fourth, how to assign it inside one tenant or publish it across tenants.

GreenGrid appears at the end of Develop as a worked illustration. The production and distribution guidance remains product-neutral.

The official starter kit uses TypeScript and React for the publisher-hosted frontend that Fabric loads in an iframe. The backend is optional and can use any HTTPS-capable stack. Python is used here for service examples.



Choose your path

The primary audience is an ISV or SDC team deciding whether to build, operate, and distribute a Fabric workload. Product and engineering leads can use the business and architecture sections. Developers can follow the implementation path. Security, operations, and publishing owners can use the production track and appendices.

The manuscript assumes familiarity with web applications, REST and JSON, OAuth concepts, source control, and basic Fabric workspaces and items. TypeScript and React help with the starter kit, while Python is needed only for the optional service examples.

By the end, the reader should be able to decide whether a workload fits the product, explain each browser, Fabric, and publisher boundary, build and diagnose a first item, define a production release unit, and choose an internal or cross-tenant distribution path.

Choose the path that matches the decision in front of you:

Path	Read	Outcome
Fast track	0 → 2 → 5 → 6 → 9	Follow the first-workload path, understand the runtime, build an item, and connect the GreenGrid example
Production track	Sections 10 through 17	Turn the prototype into a hosted, secured, packaged, assigned, and supportable product
Product and business	1 → 16 → 18	Decide who the workload serves, how it reaches customers, and what a pilot must prove

Code snippets focus on the concept being explained. Where a simplification affects security, reliability, or deployment, the limitation is stated beside the example. Version-specific details and source references are collected in the appendices.

0. Before you begin

The hands-on path assumes web hosting and HTTPS, REST and JSON, OAuth and OpenID Connect concepts, source control, and basic Fabric workspaces and items.

Scenario	Required environment	Actor or permission	Observable readiness
Common	Entra tenant, Fabric workspace on a supported F, P, or Trial capacity, PowerShell 7, Node.js, .NET SDK, Azure CLI, editor	Rights to use the workspace and sign in through the setup scripts	Repository cloned and tool versions available
Local frontend	Tenant settings enabled, personal Fabric Developer Mode, browser Local Network Access	Tenant admin for settings, workspace/capacity admin for access, developer for local mode	Dev Server and Dev Gateway run, and Hello World opens
Optional backend	HTTPS-capable runtime and identity configuration	Publisher engineering team	Health endpoint responds and token validation rejects an invalid audience
Azure hosting	Azure subscription, target resources, deployment identity	Azure subscription/resource owner	Frontend and backend deploy under the intended verified domain
Public publication	Publisher-named workload, verified publisher, support/privacy/terms evidence, Marketplace offer	Fabric publisher, tenant admin, Partner Center roles	Package reaches the intended selected, Preview, or GA stage

Tenant administrators enable settings, workspace or capacity administrators control access, publishers manage applications and packages, and standard users exercise the item.

Install the base Python dependencies only if you follow the service examples:

```
python -m venv .venv
# Windows: .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install fastapi "uvicorn[standard]" pydantic \
    "pyjwt[crypto]" requests pandas pytest
```

Add the Azure packages only for the identity, Key Vault, and telemetry examples:

```
pip install azure-identity azure-keyvault-secrets \
    azure-monitor-opentelemetry
```

The TypeScript examples run in the browser through the toolkit SDK. The Python examples run in publisher-hosted services over HTTPS.

First run

After the common and local prerequisites are ready:

1. Clone the toolkit and checkout the pinned commit.
2. From scripts/Setup, run `pwsh ./SetupWorkload.ps1 -WorkloadName "Org.YourWorkload"`.
3. From scripts/Run, start `StartDevServer.ps1`, then start `StartDevGateway.ps1` in a second terminal.
4. Enable personal Fabric Developer Mode and open the Hello World item in the development workspace.

At the pinned commit, `SetupWorkload.ps1` passes an unsupported `-Force` parameter to the package build. Check the current script before onboarding a developer. Appendix B contains the exact signatures, platform notes, and expected outputs.

If Hello World does not load, check the tenant settings, browser Local Network Access, environment files, and both local processes before changing code.

Common pitfalls

Treat early symptoms as hypotheses, not diagnoses:

Symptom	First discriminating test	Continue in
Workload does not appear	Confirm capacity, tenant settings, assignment, and local registration independently	Sections 5 and 15
Blank item surface	Inspect iframe/network loading, browser console, route, and CSP before changing identity	Sections 8 and 10
401 response	Separate missing authentication, rejected token claims, insufficient authorization, and transport failure	Sections 4 and 8
Unexpected version	Disconnect Dev Gateway, check the active package, and verify assignment scope	Sections 8, 15, and 17
Duplicate-version upload error	Compare the package version with the published-version list	Sections 13 and 17

Appendix E expands this into a boundary-based diagnostic reference with expected observations and next steps.

Understand

Key takeaways

- A workload gives an ISV or SDC a product surface inside Fabric while its SaaS, PaaS, and intellectual property stay in the publisher's cloud.
- It can connect that service to Fabric and OneLake data, but every transfer to publisher-hosted code remains the publisher's responsibility.
- Native integrations such as catalog, monitoring, Git, and deployment must be configured and validated. They are not inherited automatically.
- The package contains a workload manifest, a product manifest, and paired XML and JSON definitions for each item type.
- Reach for a workload when a user would create and open it as their own object, and a lighter extensibility point otherwise.

1. What a workload is, and why an ISV would build one

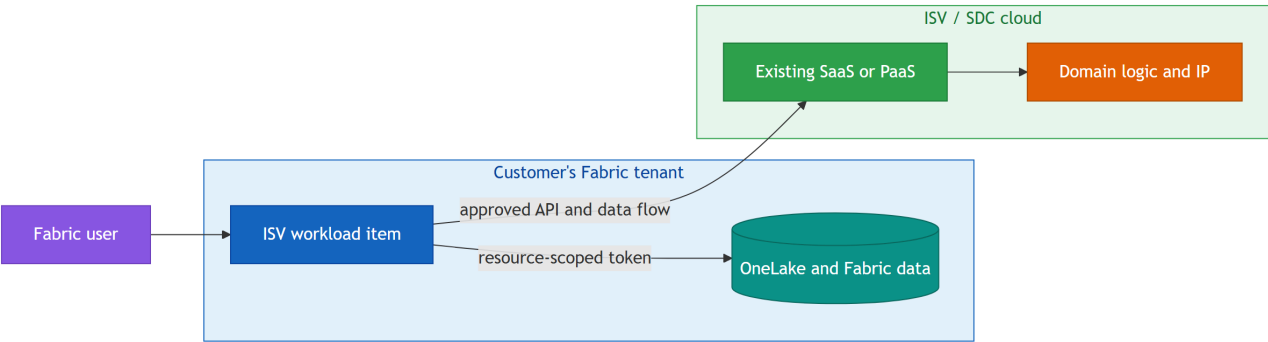
Most ISVs and SDCs do not start from an empty repository. They already have a service, an API, domain logic, support processes, and customers. The question is whether that product should remain a separate destination or become part of the place where Fabric users already work with data. A workload provides the second option.

GreenGrid provides the running example: an energy-scoring SaaS that works with approved OneLake data while its scoring logic stays in the publisher's cloud. Section 9 turns that scenario into a worked implementation.

1.1 The ISV business case: bring your service to Fabric data

For an ISV, a workload creates a route to customers whose data and analytics teams already work in Fabric. Users can configure the solution, work with approved data, and inspect results from a workspace they already use. The commercial opportunity is to reduce the effort required to demonstrate value and make adoption easier for the customer segment the product serves.

An established vendor can reuse its service and domain logic behind a Fabric-facing experience. An SDC may start with a solution it has built repeatedly for clients. Turning that work into a reusable workload can support implementation services, maintenance, and recurring licensing, but it also requires product ownership, versioning, documentation, and support.



The investment should be tied to a customer problem and measured through a pilot:

Signal	What it tests
Time to the first useful result	Whether the Fabric experience makes the product easier to evaluate
Integration effort per deployment	Whether the item and its contracts reduce repeated customer work
Active use after the pilot	Whether the workload becomes part of a recurring workflow
Pilot conversion and support cost	Whether adoption can become a sustainable product motion

If customers only need an API connection or a scheduled transformation, these gains may not justify a dedicated workload. Fabric distribution and Microsoft Marketplace can support discovery, evaluation, and acquisition, depending on the publishing and offer model. Microsoft co-sell is a separate commercial process with its own [eligibility requirements](#). The publisher still owns customer targeting, pricing, and the work required to convert interest into adoption.

1.2 What Fabric gives you, and what a workload adds

Fabric is organized around workspaces and items, with OneLake as the shared data plane. A workload adds a publisher-defined item and experience to that model. The user gains one place to configure the product, work with permitted data, and return to its results.

Catalog, monitoring, Git, deployment, jobs, lifecycle, and protection still require their own declarations or implementation. Appendix A lists those capabilities and their limits.

1.3 The toolkit, when to use it, and its limits

The Extensibility Toolkit is the supported starting point for new workloads. It provides a starter project, the workload client SDK, setup and build scripts, and the local Dev Server and Dev Gateway loop. It is the current evolution of the older Workload Development Kit.

It fits when users need a custom item with its own editor and lifecycle: a domain-specific authoring tool, governance console, industry workflow, or operational application. The baseline is frontend-first. Remote hosting exists in the toolkit, but its publication status must be confirmed for the intended stage. Appendix F records the current source discrepancy.

The toolkit does not run arbitrary server code inside a general-purpose Fabric runtime, bypass capacity quotas, or remove the publisher's responsibility for hosting and support. Development requires a supported F, P, or Trial capacity. Power BI Pro alone is not enough.

Other extensibility points cover narrower needs. A visualization inside a report is a Power BI custom visual. A rule that watches a stream and reacts to a condition belongs in Activator. A scheduled transformation may belong in a pipeline, notebook, or Azure service. Use a workload when the customer should create and open a distinct item with its own editor and lifecycle.

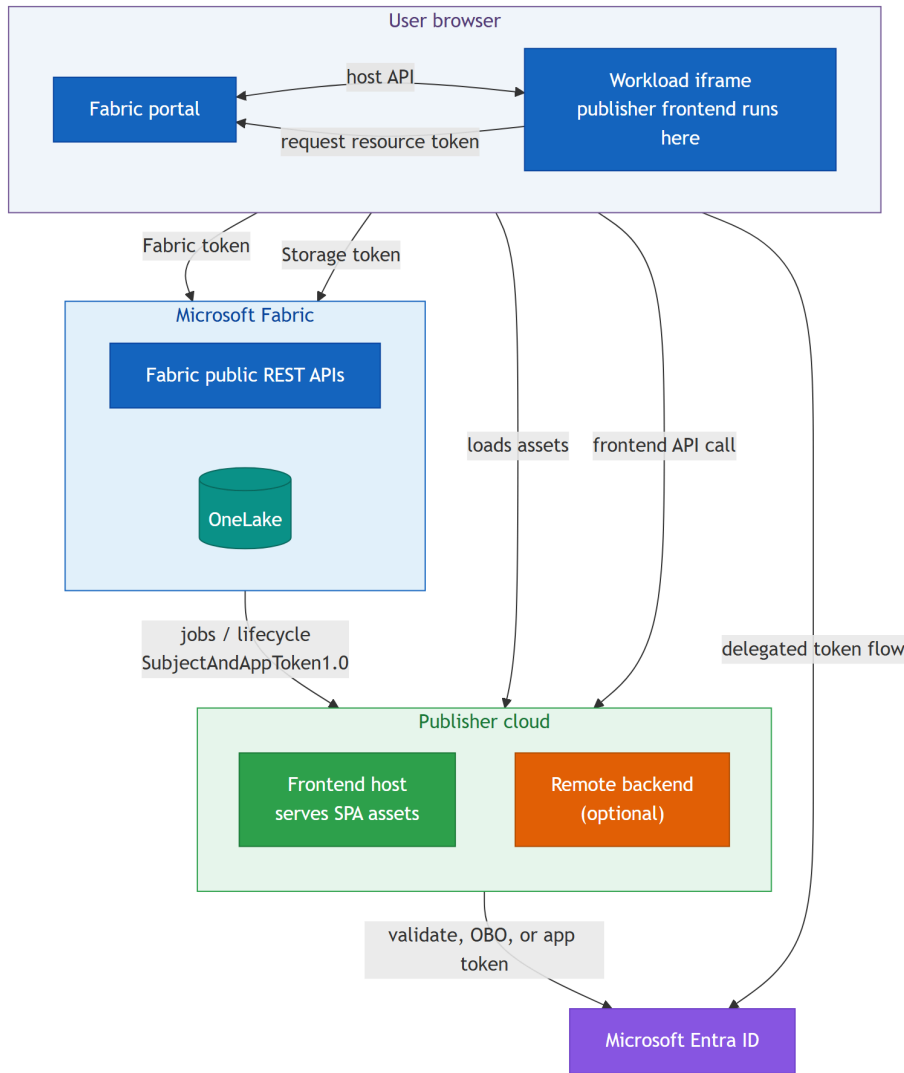
Option	Choose it when	Ongoing cost
Fabric workload	Users need a persistent workspace item, dedicated editor, item lifecycle, and a route to multiple Fabric tenants	Frontend and SDK compatibility, identity, publishing review, consent changes, support, and optional backend operations
Power BI custom visual	The experience belongs inside a report visual	Visual certification, report compatibility, and visual-specific UX
Activator	The need is an event-driven rule, alert, or action	Event model, rule operations, and response handling
Pipeline, notebook, or function	The primary job is transformation, orchestration, or scheduled compute	Data and compute operations without a custom item editor
Separate SaaS experience	The product does not need a Fabric item or native lifecycle	Separate navigation, identity context, integration, and customer administration

A workload is justified when the persistent item and Fabric-native workflow create enough value to cover its long-term compatibility, security, publishing, and support costs. If the requirement is only a chart, rule, or background transformation, the lighter option is usually easier to operate.

2. How a workload runs: architecture, the host, and one request

2.1 The Fabric host, your frontend, and an optional backend

Four boundaries matter when a workload is open. The user's browser runs the Fabric portal. The portal creates an iframe and loads the publisher's frontend from the endpoint declared in the workload manifest. The JavaScript executes inside that iframe and communicates with Fabric through the workload client SDK. Fabric APIs and OneLake remain separate resource endpoints. A publisher backend is optional and runs outside Fabric on infrastructure the publisher operates.



These relationships use different contracts:

Relationship	Caller	Authentication contract	Typical purpose
Frontend → Fabric or OneLake	Workload iframe	Delegated resource token from <code>acquireFrontendAccessToken</code>	User-authorized data and item operations
Frontend → publisher API	Workload iframe	Delegated token whose audience is the publisher API	Product-specific frontend request
Fabric → publisher remote endpoint	Fabric service	<code>SubjectAndAppToken1.0</code> , with a required app token and an operation-dependent subject token	Jobs and item lifecycle callbacks
Publisher backend → target resource	Publisher service	OBO, backend application, or managed identity according to the target	Downstream user or service operation

Section 4 defines the token and header rules for these calls. Appendix F records the version-specific sources and discrepancies.

The manifest tells Fabric where to load the frontend and which item capabilities exist. The SDK provides the supported bridge for theme, navigation, dialogs, notifications, item operations, and token acquisition. Code inside the iframe should use that contract rather than reaching into the portal DOM or assuming internal Fabric routes.

2.2 Items as native artifacts, with explicit integrations

An item type gives the workload a durable object in a workspace. Its instances can use Fabric item APIs and workspace permissions, but other integrations depend on explicit declarations and implementation. Appendix A contains the capability matrix and sources. Treat that matrix as part of the product design, not as a blanket promise that every custom item behaves exactly like every built-in item.

Stable concept: keep control plane and data plane separate.

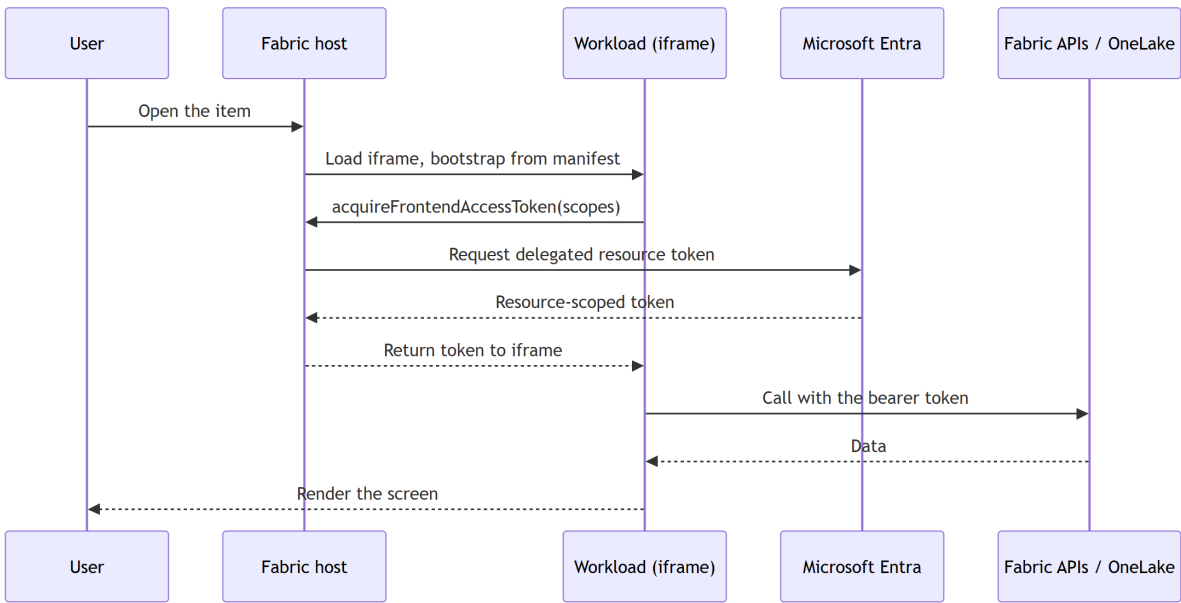
The item definition belongs to the control plane. Current [definition guidance](#) recommends a reasonable number of human-readable text parts, up to five. It does not document five as an enforced API rejection threshold. Fabric does not schema-validate custom part contents for you. Those parts can support Git integration and deployment because they are compact and reviewable. Files, tables, model outputs, and other large data belong in OneLake or another declared data store.

Concern	Control plane	Data plane
Purpose	Describe and configure the item	Hold business data and large results
Typical content	Small text definition parts, identifiers, settings, schema version	OneLake files and tables, or an explicitly governed publisher store
Delivery	Fabric item APIs, Git, and deployment where supported	Data-engineering and application data paths
Design rule	Human-readable, reviewable, no secrets	Access, residency, retention, and scale designed explicitly

That separation also clarifies ownership. Fabric stores and moves the definition through its item APIs. The workload code interprets it. The publisher remains responsible for any external state held by its own service. If a value must follow the item through source control or deployment, it is a candidate for the definition. If it is customer data, a large result set, or shared operational state, it needs an appropriate data-plane design.

2.3 A request, step by step

The pieces are easier to hold once you follow a single direct call. A user opens an item. Fabric loads the publisher frontend in the iframe. The frontend asks the SDK for a delegated token for a specific resource and set of scopes. Fabric and Entra return that resource-scoped token, and the frontend calls the matching Fabric or OneLake endpoint. If the frontend sends data or a token to the publisher backend, that is a separate boundary with its own disclosure, retention, and security obligations.



Use the sequence as a diagnostic map: open, bootstrap, token, call, render. Section 8 shows how to identify the last step that produced evidence without treating one symptom as proof of one cause.

3. The manifest package: the contract with Fabric

3.1 Workload, product, and paired item manifests

A workload package contains several declarations with different jobs. `WorkloadManifest.xml` defines the workload identity, hosting mode, Entra applications, and service endpoints. `Product.json` holds product-level presentation and support metadata. Each item type then has two files: an XML platform definition and a JSON frontend definition.

The pinned toolkit uses this source layout:

```

Workload/Manifest/
├─ WorkloadManifest.xml
├─ Product.json
├─ ManifestPackage.nuspec
├─ *.xsd
├─ assets/
├─ images/
├─ locales/
├─ items/
├─ ForecastItem/
├─ ForecastItem.xml
├─ ForecastItem.json

```

The build selects and flattens the manifest files into two package segments:

The build flattens those files into the following .nupkg payload. NuGet bookkeeping is omitted:

```

BE/
├─ WorkloadManifest.xml
├─ ForecastItem.xml
FE/
├─ Product.json
├─ ForecastItem.json
├─ assets/
├─ images/
├─ locales/

```

BE and FE are manifest segments, not hosted backend and frontend binaries. The frontend bundle is built and released separately. The pinned release script copies it to release/app. Application source, remote-service code, environment files, XSDs, nested source folders, business data, and item-instance definitions are not workload payloads in the .nupkg. The [manifest package contract](#), [pinned nuspec](#), [package build](#), and [separate application release](#) define the boundary.

The XML and JSON item manifests are a pair, not interchangeable versions of the same file. The XML side describes the item type to Fabric's platform services. The JSON side describes frontend behavior and presentation. The current schema and starter kit remain the authority for exact fields because remote hosting, lifecycle, and publishing support continued to change during 2026.

Do not confuse item-type manifests with an item-instance definition:

Object	Level	Primary owner	Delivery or storage	Changed when
WorkloadManifest.xml	Workload release	Publisher	Manifest package	Hosting, identity, endpoints, or workload version changes
Product.json	Product release	Publisher	Manifest package	Product metadata, support, listing, or presentation changes
{Item}.xml	Item type	Publisher	Manifest package	Platform capabilities or contracts change
{Item}.json	Item type	Publisher	Manifest package	Frontend behavior or presentation changes
Item definition parts	Item instance	Fabric stores the parts. The publisher defines the schema	Fabric item-definition APIs and supported ALM paths	A user configures an item or the definition schema migrates

The item-definition API represents one instance as parts with path, payload, and payloadType. Workload-owned parts hold functional state, while Fabric can own platform material such as .platform. Those parts are not the XML/JSON item-type manifest pair. See the [item-definition contract](#).

A simplified frontend-hosting manifest looks like this. It omits required fields, so start from the current starter-kit manifest:

```
<WorkloadManifestConfiguration SchemaVersion="2.0.0">
  <Workload WorkloadName="Org.YourWorkload" HostingType="FERemote">
    <Version>1.0.0</Version>
    <RemoteServiceConfiguration>
      <CloudServiceConfiguration>
        <AADFEApp><AppId>00000000-0000-0000-0000-000000000000</AppId></AADFEApp>
        <Endpoints>
          <ServiceEndpoint>
            <Name>Frontend</Name>
            <Url>https://your-frontend.example.com</Url>
          </ServiceEndpoint>
        </Endpoints>
      </CloudServiceConfiguration>
    </RemoteServiceConfiguration>
  </Workload>
</WorkloadManifestConfiguration>
```

Package limits affect item count, asset count and size, `Product.json`, and item filenames. Appendix A keeps the current values and sources in one place.

The package declares what Fabric needs to load and present the workload. It does not contain the running frontend or backend, which remain at the declared publisher endpoints. Every upload needs a package version Fabric has not seen before.

3.2 Identity and naming

The workload name is the identifier Fabric uses to register the workload, and its form is a decision about distribution that you make early, because it is baked into the package. The current [publishing overview](#) defines the internal and cross-tenant scenarios.

Aspect	Org. [Name] (internal)	[Publisher].[Workload] (cross-tenant)
Audience	Your own tenant	Other tenants
Registration	No separate workload-name registration	Name and publishing tenant reserved permanently on first confirmation
Availability	Upload, activate, then assign to tenant, capacity, or workspace	Selected tenants, then Preview, then GA
Publishing requirements	General hosting, manifest, identity, and tenant requirements	General, workload, item, attestation, support, and public-review requirements
Commercial setup	Not required for an internal workload	Microsoft Marketplace SaaS offer required for Preview and GA
Publisher identity	Verified domain	Verified domain plus Microsoft verified publisher requirements
Name length	No separate limit established by the cited publishing overview	Workload portion at most 32 characters

A publisher name is reserved permanently when it is confirmed during the first upload. The publishing tenant is fixed as well. Treat that confirmation as a product decision, not a temporary test value. Moving later from an Org. * identity to a publisher identity affects the package, application registrations, endpoints, documentation, and installed tenants, so plan the public naming path before customers depend on the internal one.

4. Identity and access with Microsoft Entra

4.1 Delegated frontend tokens and backend OBO

The workload client exposes one acquisition method, but it does not produce one universal bearer token. The frontend calls `acquireFrontendAccessToken` with the scopes for a specific resource. The documented result object exposes the bearer value through its `token` property. Fabric APIs, OneLake Storage, Microsoft Graph, and a publisher API have different audiences, so each needs a token requested for its own scopes.

The client below shows resource-specific acquisition. Consent handling, retries, caching, and resource authorization remain application concerns.

```
import { WorkloadClientAPI } from "@ms-fabric/workload-client";

export async function callResource(
  client: WorkloadClientAPI,
  url: string,
  scopes: string[],
): Promise<unknown> {
  const tokenResult = await client.auth.acquireFrontendAccessToken({ scopes });
  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${tokenResult.token}` },
  });
  if (!res.ok) throw new Error(`Resource call ${res.status}`);
  return res.json();
}
```

That token is a delegated frontend token. It represents the signed-in user for the requested resource and scopes. The frontend does not receive the user's password, but it does receive an access token and can send it to the target service. If the target is a publisher backend, the publisher now handles that token and must protect it accordingly.

On-behalf-of (OBO) has a narrower meaning. It is the backend exchange in which a remote service accepts a user's subject token and obtains a new token for another resource. Do not call the frontend token itself an OBO token, and do not forward a Fabric token to OneLake or another API with a different audience.

4.2 Entra applications, scopes, and service identities

Every workload uses Entra application registrations. `Fabric.Extend` remains mandatory. The exact setup then depends on the hosting mode. The current frontend-hosting setup configures the `/close` redirect and workload sign-in redirects for both Fabric and Power BI portal domains. A remote backend adds its own application, exposed scopes, preauthorization, and OBO or service-to-service configuration. Follow the setup script and current authentication guidance rather than copying one redirect list between modes.

Define narrow scopes for a publisher API, such as separate read and write operations, and validate them on the server. The following dependency illustrates a single-tenant internal API. A cross-tenant service also needs an issuer and tenant-validation strategy. Do not reuse a fixed-tenant validator for public distribution.

Token version determines the audience contract. For a v2 access token, `aud` is the client ID of the target API. A v1 token can use a client ID or resource URI according to the resource configuration. Pair the expected audience with the matching issuer and version instead of treating the Application ID URI as a universal audience. See the [Entra access-token claims reference](#).

The validator below is deliberately single-tenant and v2-specific. A production service also needs tenant policy, resource authorization, key-cache behavior, telemetry, and security review.

```
# auth.py - validate a Fabric/Entra bearer token in a Python backend.
import os
import jwt
import jwt
from jwt import PyJWKClient
from fastapi import Header, HTTPException

TENANT_ID = os.environ["TENANT_ID"]
# For a v2 token issued to this custom API, aud is the API application's client ID.
API_CLIENT_ID = os.environ["API_CLIENT_ID"]
ISSUER = f"https://login.microsoftonline.com/{TENANT_ID}/v2.0"
JWKS_URL = f"https://login.microsoftonline.com/{TENANT_ID}/discovery/v2.0/keys"

_jwks = PyJWKClient(JWKS_URL) # caches signing keys

def require_scope(required: str):
    def dependency(authorization: str = Header(...)) -> dict:
        if not authorization.startswith("Bearer "):
            raise HTTPException(401, "Missing bearer token")
        token = authorization.split(" ", 1)[1]
        try:
            signing_key = _jwks.get_signing_key_from_jwt(token).key
            claims = jwt.decode(
                token, signing_key, algorithms=["RS256"],
                audience=API_CLIENT_ID, issuer=ISSUER,
            )
        except jwt.PyJWTError as exc:
            raise HTTPException(401, "Invalid bearer token") from exc
        scopes = claims.get("scp", "").split()
        if claims.get("ver") != "2.0":
            raise HTTPException(401, "Unexpected token version")
        if required not in scopes:
            raise HTTPException(403, "Required scope is missing")
        return claims
    return dependency
```

Use the dependency at the route boundary:

```
from fastapi import FastAPI, Depends
from auth import require_scope

app = FastAPI()

@app.get("/forecasts")
def list_forecasts(claims: dict = Depends(require_scope("data.read"))):
    user = claims.get("oid") # the signed-in user's object id
    return {"user": user, "forecasts": []}
```

Validating the token and scope authenticates the caller and constrains the delegated operation. It does not prove that the caller may access a particular tenant, workspace, item, or publisher-side record. Apply resource authorization after token validation.

Three identities may therefore appear in one product: the signed-in user represented by a delegated token, the backend application used for remote workload and OBO flows, and a managed identity used by the publisher's Azure-hosted service for supported service-to-service calls. They are not interchangeable. Section 11 separates them in the production design, and section 12 covers what happens when data or tokens cross into publisher-operated systems.

Boundary	Identity represented	Expected audience	Server checks	Resource authorization
Frontend → Fabric API	Signed-in user	Fabric resource	Microsoft resource validation	Fabric workspace/item permissions
Frontend → OneLake	Signed-in user	Storage / OneLake resource	Microsoft resource validation	OneLake data permissions
Frontend → ISV API with v2 token	Signed-in user	ISV API client ID	Signature, v2 issuer, client-ID audience, expiry, scope	Publisher tenant/workspace/item or product policy

Boundary	Identity represented	Expected audience	Server checks	Resource authorization
Fabric → publisher remote endpoint	Fabric app plus operation-dependent subject	Backend endpoint contract	SubjectAndAppToken1.0, required app token, optional subject token, operation, tenant and correlation headers where specified	Publisher endpoint and item policy
Backend → Fabric or OneLake through OBO	Signed-in user	Downstream target resource	OBO acquisition and downstream validation	Fabric/OneLake permissions
Backend → service as application	Backend app or managed identity	Target service	Application token and credential/identity validation	Application permissions or RBAC

Develop

Key takeaways

- The Dev Server serves the workload from localhost and the Dev Gateway lets the Fabric portal render it.
- An item combines a control-plane definition, a frontend experience, and explicit integrations with Fabric and data services.
- The repository's AI guidance can scaffold repeatable work, but it is versioned project guidance rather than a product CLI or a source of truth.
- Diagnose by reading one boundary at a time: token, manifest, gateway, iframe.

5. The toolkit and the development environment

5.1 The starter kit, setup script, and Entra applications

Clone the starter kit, then run its setup from scripts/Setup:

```
git clone https://github.com/microsoft/fabric-extensibility-toolkit
cd fabric-extensibility-toolkit/scripts/Setup
pwsh ./SetupWorkload.ps1 -WorkloadName "Org.YourWorkload"
```

The setup creates or reuses the Entra applications and writes the local environment. Dev Gateway handles routing, not identity, so the workload still requests real delegated tokens. Appendix B lists every parameter and the known issue in the pinned package-build handoff.

5.2 Dev Server, Dev Gateway, and the Hello World checkpoint

Two long-running processes drive local development. Dev Server hosts the SPA and development manifests. Dev Gateway registers the local instance and points Fabric to Dev Server.

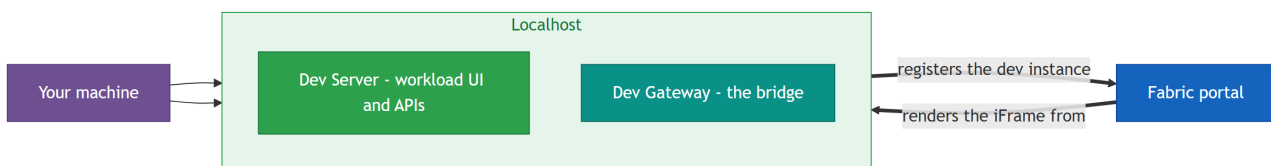
Start Dev Server in the first terminal:

```
cd scripts/Run
pwsh ./StartDevServer.ps1
```

Start Dev Gateway in the second:

```
cd scripts/Run
pwsh ./StartDevGateway.ps1
```

Appendix B records the script parameters and the Linux login flag.



Current setup guidance names three tenant settings:

1. **Capacity admins and contributors can add and remove additional workloads**
2. **Workspace admins can develop partner workloads**
3. **Users can see and work with additional workloads not validated by Microsoft**

Each developer also enables personal **Fabric Developer Mode**. The workspace must use a supported F, P, or Trial capacity. In current Chromium-based browsers, Local Network Access can block the portal-to-localhost call, so check that permission when both processes are healthy but the item stays blank.

Prove the environment with the Hello World item before adding your own type. If Hello World opens, the gateway and portal path works. If it does not, inspect the tenant settings, personal developer mode, browser local-network permission, capacity, environment files, and both terminal processes.

| 6. Building an item: editor, data, and capabilities

6.1 The item, its editor, and how it surfaces

A new item type starts from the generator in `scripts/Setup`. `ItemName` names the new type, while optional `srcItemName` selects the source item and defaults to `HelloWorld`:

```
cd scripts/Setup
pwsh ./CreateNewItem.ps1 -ItemName "Forecast" -srcItemName "HelloWorld"
```

The result is a copied item structure. Update `ITEM_NAMES`, `Product.json`, the locale entries, and `App.tsx` before opening it in Fabric. Appendix B keeps the exact script contract.

The starter kit gives you a definition model, editor, views, and ribbon actions. Keep the definition compact:

```
// ForecastItemDefinition.ts - compact, human-readable control-plane configuration.
export interface ForecastItemDefinition {
  horizonDays: number;
  sourceItemId?: string;
  schemaVersion: number;
}

export const DEFAULT_FORECAST: ForecastItemDefinition = {
  horizonDays: 14,
  schemaVersion: 1,
};
```

Fabric provides the item creation surface. The editor route maps the Fabric item identifier to your component:

```
// App.tsx - register the editor route so the host's navigation matches a component.
<Route path="/forecast-editor/:itemObjectId">
  <ForecastItemEditor workloadClient={workloadClient} />
</Route>
```

The starter kit updates definition parts through the item CRUD API. Verify the exact types in the installed SDK:

```
import { WorkloadClientAPI } from "@ms-fabric/workload-client";
import { ForecastItemDefinition } from "./ForecastItemDefinition";

export async function saveForecast(
  client: WorkloadClientAPI, itemId: string, def: ForecastItemDefinition,
) {
  const parts = serializeDefinition(def);
  await client.itemCrud.updateItemDefinition(itemId, parts);
}
```

The exact argument shape belongs to the installed SDK. The example shows the boundary rather than replacing that typed contract. For the type to appear in the portal, the item XML, item JSON, product metadata, locale entries, routes, and packaged assets must agree. A sensitivity label is not automatically applied simply because the item uses the standard creation surface.

Keep one development item open while Dev Server watches the frontend. That gives you a short edit-and-refresh loop without creating a new item for every UI change.

6.2 Separating item definitions from OneLake data

A workload often needs data from OneLake. Keep the data source behind a function so the screen does not care whether development uses seed data or a real file. Request a Storage-scoped delegated token for OneLake. Do not reuse a Fabric API token with a different audience.

In the browser, the frontend reads a OneLake file with the user's Storage-scoped token:

```
// Read a CSV from OneLake in the frontend, as the signed-in user.
export async function readSitesCsv(
  client: WorkloadClientAPI, workspaceId: string, lakehouseId: string,
): Promise<string> {
  const tokenResult = await client.auth.acquireFrontendAccessToken({
    scopes: ONELAKE_STORAGE_SCOPES,
  });
  const url =
    `https://onelake.dfs.fabric.microsoft.com/${workspaceId}/` +
    `${lakehouseId}/Files/sites.csv`;
  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${tokenResult.token}` },
  });
  if (!res.ok) throw new Error(`OneLake ${res.status}`);
  return res.text();
}
```

When the work belongs on a server, the backend normally exchanges the incoming user token through OBO for a OneLake token. The reader below assumes that exchange has already happened. Production code also needs retries, streaming, and large-file handling:

```
# onelake.py - read a OneLake CSV in a Python backend, as the calling user.
import io
import pandas as pd
import requests

ONELAKE = "https://onelake.dfs.fabric.microsoft.com"

def read_sites(onelake_token: str, workspace_id: str, lakehouse_id: str) -> pd.DataFrame:
    url = f"{ONELAKE}/{workspace_id}/{lakehouse_id}/Files/sites.csv"
    resp = requests.get(url, headers={"Authorization": f"Bearer {onelake_token}"}, timeout=30)
    resp.raise_for_status()
    return pd.read_csv(io.StringIO(resp.text))
```

The item definition can hold small settings such as a horizon, a source item identifier, and a schema version. It should not contain copied table data, large results, secrets, or opaque binary state. Those belong in OneLake or a publisher store with an explicit data-governance model.

6.3 Remote jobs, lifecycle events, and other capabilities

A settings dialog, About page, localization, catalog metadata, and monitoring integration each require their own manifest or frontend work. Add only the capabilities you implement and test.

A remote job is declared through the item manifest. Fabric schedules and monitors it, while the publisher endpoint executes the work and reports status. Cancellation, retry, detail views, and Recent Runs need additional declarations or handlers. The exact job route still differs across first-party samples, so confirm it for the intended publication stage. Appendix A covers capabilities, and Appendix B covers the remote-hosting script.

The Python below shows execution logic, not the Fabric remote-job protocol. Its in-memory status store is unsuitable for production:

```
# jobs.py - a server-side job the workload starts and polls.
from fastapi import FastAPI, BackgroundTasks
from uuid import uuid4

app = FastAPI()
_status: dict[str, dict] = {}

def run_forecast(job_id: str, horizon_days: int) -> None:
    _status[job_id] = {"state": "running", "progress": 0}
    for step in range(horizon_days):
        ... # compute one day of the forecast
        _status[job_id] = {"state": "running",
                           "progress": round((step + 1) / horizon_days * 100)}
    _status[job_id] = {"state": "succeeded", "progress": 100}

@app.post("/jobs/forecast")
def start(horizon_days: int, tasks: BackgroundTasks) -> dict:
    job_id = str(uuid4())
    _status[job_id] = {"state": "queued", "progress": 0}
    tasks.add_task(run_forecast, job_id, horizon_days)
    return {"jobId": job_id}

@app.get("/jobs/{job_id}")
def status(job_id: str) -> dict:
    return _status.get(job_id, {"state": "unknown"})
```

Remote lifecycle notifications extend beyond create, update, and delete. Current guidance covers soft delete, hard delete, and restore. If OnDelete is enabled, the publisher must also implement restoration handling, and delete notifications may arrive without a subject token. Create, update, or restore can be rejected in documented cases. Deletion cannot be blocked. Treat these operations as a durable service contract, not a frontend callback.

7. Developing with AI assistance

The toolkit repository includes context files, task procedures, Copilot instructions, and an @fabric agent. They help an assistant navigate the project, but the checked-out SDK types, manifests, scripts, and current Microsoft Learn pages remain the contracts to verify.

Stable concept: AI output is a proposal, not platform truth.

Use a short reviewable workflow:

Step	Developer action	Expected result
Frame	State the item, user task, affected boundary, and non-goals	A bounded implementation task
Ground	Point to repository instructions, current SDK types, scripts, and Microsoft Learn	The agent names the contracts it will use
Bound	Limit files, tools, permissions, and allowed side effects	A reviewable change rather than a repository-wide rewrite
Review	Inspect the diff, data flows, scopes, error behavior, and generated assumptions	Product and security decisions remain human-owned
Verify	Run the four gates below and record the evidence	A result that can be accepted or rejected

The four gates are straightforward: check the SDK method and type, confirm that every manifest and route agrees, run the real build scripts, then open the item in Fabric and exercise its identity and error paths.

7.1 Repository guidance: useful, mutable, and versioned

The .ai/ files explain the platform, project structure, and common tasks. They are guidance, not executable product commands. If an instruction names a command that the repository does not ship, use the actual script or documented API. Appendix C contains the file map and authority order.

7.2 The Copilot agent and the optional community UX MCP

Ask for a bounded repository change and name the expected verification:

```
@fabric create a Forecast item type with a horizon setting.  
Use the current item generator and manifest pair, update localization and routes,  
then run Dev Server and Dev Gateway and verify the item opens in Fabric.
```

The agent can scaffold and wire the item, but the developer still owns the data contract, permission model, and user experience. The optional Fabric UX MCP referenced by the toolkit is community-maintained. Its configuration and review points are in Appendix C.

7.3 What it generates, and keeping it honest

An agent can scaffold manifests, editor views, routes, locale entries, and token calls. Accept the change only when the item opens through Dev Gateway, the package passes local checks, the frontend requests the right audience, and the diff preserves the intended data and permission boundaries.

| 8. Diagnostics and debugging

A workload is a chain of boundaries. Diagnose it from observed requests, responses, and browser errors rather than from symptom shortcuts.

8.1 Reading the chain, and the token and manifest boundaries

The reference path is the one Hello World proved: Dev Server, Dev Gateway, the Fabric host, and the iframe. Use it as a comparison, not a verdict. If Hello World opens, the local path is more likely to be healthy, but the new item can still fail in routing, manifests, JavaScript, identity, authorization, transport, or data access.

For token failures, confirm scopes, audience, tenant and issuer policy, expiry, and the represented identity. Log the validation category on the server without logging the token or personal data:

```
# Log validation detail server-side; return a generic error to the caller.  
import logging  
  
logger = logging.getLogger("workload.auth")  
  
try:  
    claims = jwt.decode(token, signing_key, algorithms=["RS256"],  
                        audience=AUDIENCE, issuer=ISSUER)  
except jwt.ExpiredSignatureError:  
    logger.info("token expired")  
    raise HTTPException(401, "Invalid bearer token")  
except jwt.InvalidAudienceError:  
    logger.warning("token audience mismatch")  
    raise HTTPException(401, "Invalid bearer token")  
except jwt.InvalidIssuerError:  
    logger.warning("token issuer rejected")  
    raise HTTPException(401, "Invalid bearer token")
```

The manifest boundary turns on declarations agreeing across `WorkloadManifest.xml`, `Product.json`, the paired item files, locale entries, routes, and assets. `BuildManifestPackage.ps1 -ValidateFiles $true` performs local XML and XSD checks while building the package. The separate publishing validator runs against a workload already published to a tenant and does not guarantee Microsoft approval. Treat these as two different gates.

Use one discriminating test at a time:

1. Record the observable signal: HTTP status, console error, missing request, empty response, or stale UI state.
2. List the boundaries that can produce that signal.
3. Choose a test that changes one hypothesis and define the expected observation.
4. Use the result to select the next boundary instead of changing several layers together.

Appendix E provides the symptom, plausible causes, first test, expected observation, and next step for the common paths.

8.2 The Dev Gateway, the iframe boundary, and correlation

Dev Gateway registers the local workload and points Fabric at Dev Server. It does not provide identity. While it is connected, the local workload can take precedence over an uploaded version for the configured workspaces. A blocked browser local-network permission can look like a stopped gateway, so test both before changing application code.

Use the host SDK and keep loading, empty, and error states distinct:

```
// Keep loading, empty, and failed requests distinct.
if (error) return <MessageBar intent="error">Could not load: {error}</MessageBar>;
if (!data) return <Spinner label="Loading..." />;
if (data.length === 0) return <EmptyState title="Nothing here yet" />;
return <DataGrid rows={data} />;
```

Fabric-initiated operations can carry ActivityId and RequestId. Propagate them when present. Browser calls created by workload code need their own correlation:

```
# Propagate documented Fabric correlation IDs without logging request data.
import logging
from fastapi import Request

logger = logging.getLogger("workload")

@app.middleware("http")
async def correlate(request: Request, call_next):
    activity_id = request.headers.get("ActivityId", "-")
    request_id = request.headers.get("RequestId", "-")
    logger.info("request start", extra={"activity_id": activity_id, "request_id": request_id})
    response = await call_next(request)
    logger.info("request end status=%s", response.status_code,
               extra={"activity_id": activity_id, "request_id": request_id})
    return response
```

9. Illustration: GreenGrid

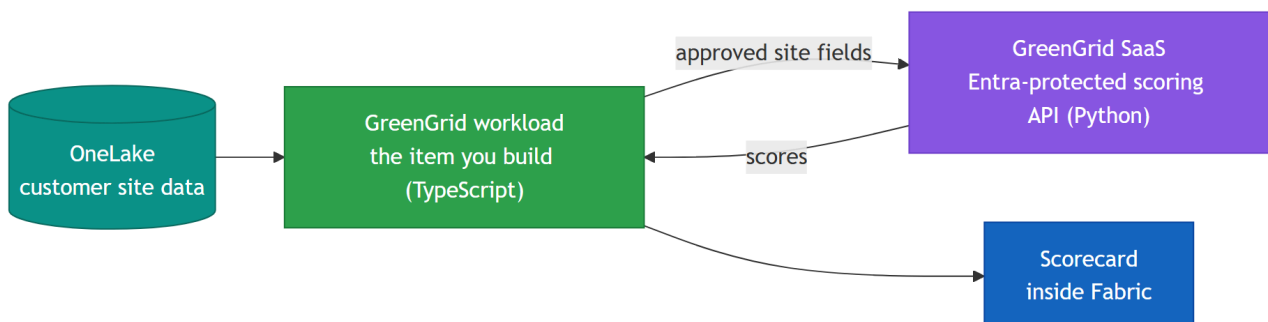
This is a teaching sample. It isolates the development concepts and deliberately skips production concerns such as a verified domain, a managed identity, and a release pipeline. Read it as a way to see the previous sections in motion, not as a blueprint to copy into production.

Reference implementation: [fredgis/FY27FabricMotion](#).

9.1 What GreenGrid is

GreenGrid Analytics is a fictional ISV and software development company. Its product scores the sustainability of customer sites from energy use and renewable mix. The source data lives in the customer's OneLake, while GreenGrid's scoring logic remains in an Entra-protected SaaS API. The workload reads the approved fields, sends them to that API, and renders the returned scores inside Fabric.

That API call is a real publisher boundary. Selected site data leaves Fabric and reaches GreenGrid's service, so the product must document the fields it sends, where they are processed, how long they are retained, and which subprocessors are involved. The example keeps the payload small, but it does not pretend that an external scoring call leaves all data inside the tenant.



The split of languages is deliberate. The algorithm is GreenGrid's product and stays on its server. The TypeScript and React frontend runs in the workload iframe. Source data remains in OneLake, while the approved request payload crosses to the scoring service.

The result is the GreenGrid Scorecard item shown below, open in a Fabric workspace and scoring five sites for the fictional customer Contoso Energy.

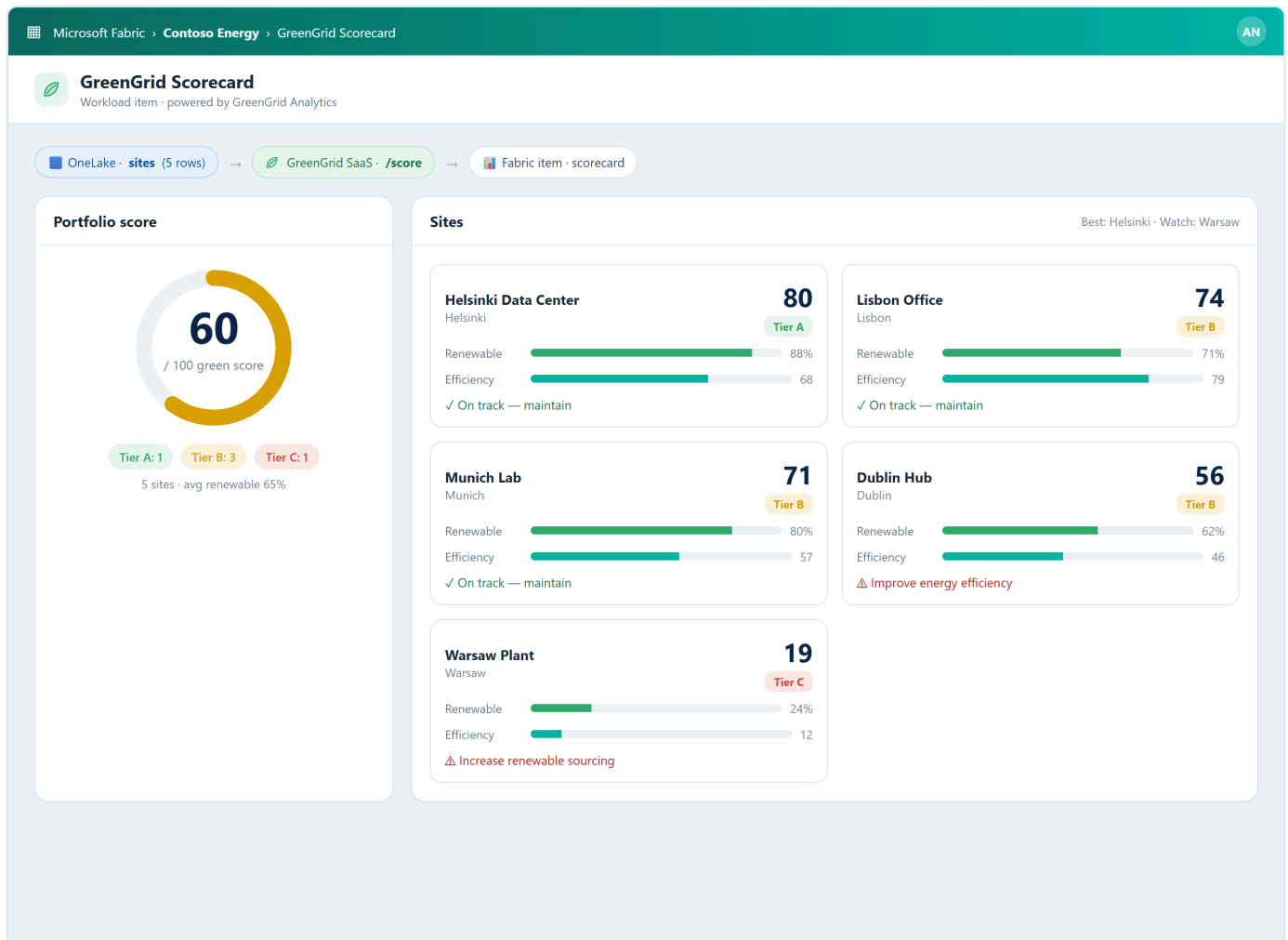


Figure 9.1 · The GreenGrid Scorecard, a custom workload item, rendered natively in the Fabric portal. The breadcrumb, the chrome, and the theme belong to Fabric. The screen belongs to GreenGrid.

The numbers on that screen come from the Python service and the seed data used in section 9.3. The portfolio averages 60 out of 100. Helsinki scores 80 and lands in Tier A on an 88 percent renewable mix. Warsaw scores 19 and falls to Tier C on 24 percent. The strip across the top makes the boundary visible: OneLake supplies the approved fields, GreenGrid scores them, and the item renders the result. The experience sits inside Fabric even though the algorithm remains in the publisher's service.

9.2 The scoring service, in Python

The SaaS is a small FastAPI service protected by the `require_scope` dependency from section 4.2. It takes a list of sites and returns a score, tier, and portfolio summary. The formula is deliberately simple. The useful part is the authenticated API boundary.

The scoring formula and route are intentionally small. Production adds tenant authorization, rate limits, resilient storage, operational controls, and a complete threat model.

```
# greengrid_saas.py - the scoring algorithm, owned by GreenGrid, hosted by GreenGrid.
from fastapi import Depends, FastAPI
from pydantic import BaseModel, Field
from auth import require_scope

app = FastAPI(title="GreenGrid Scoring")

class Site(BaseModel):
    siteId: str
    name: str
    city: str
    energyKwh: float = Field(gt=0)
    renewablePct: float = Field(ge=0, le=100)

class Portfolio(BaseModel):
    sites: list[Site]

def score_one(site: Site) -> dict:
    # Efficiency rewards low energy; the green score blends it with renewable share.
    efficiency = max(0.0, 100.0 - site.energyKwh / 10.0)
    green = round(0.4 * efficiency + 0.6 * site.renewablePct)
    tier = "A" if green >= 75 else "B" if green >= 50 else "C"
    tip = ("Strong renewable mix" if site.renewablePct >= 80
          else "Shift load to renewables" if site.renewablePct < 50
          else "Trim peak consumption")
    return {**site.model_dump(), "efficiency": round(efficiency),
            "greenScore": green, "tier": tier, "tip": tip}

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}

@app.post("/score")
def score(
    body: Portfolio,
    _claims: dict = Depends(require_scope("score.run")),
) -> dict:
    scored = [score_one(s) for s in body.sites]
    avg = round(sum(s["greenScore"] for s in scored) / len(scored)) if scored else 0
    best = max(scored, key=lambda s: s["greenScore"], default=None)
    worst = min(scored, key=lambda s: s["greenScore"], default=None)
    return {"sites": scored,
            "summary": {"totalSites": len(scored), "avgScore": avg,
                       "best": best, "worst": worst}}
```

Run it locally with uvicorn `greengrid_saas:app --port 8787`. The health route confirms that the process is running:

The health route only confirms that the local process responds:

```
curl -s http://localhost:8787/health
```

The `/score` route needs a real token for GreenGrid's API scope. The workload supplies that token in the next milestone. Unit tests can exercise the pure scoring function without weakening the API:

The deterministic scoring function can be tested directly:

```
# test_scoring.py
from greengrid_saas import Site, score_one

def test_high_renewable_site_scores_tier_a():
    s = Site(siteId="s1", name="Helsinki DC", city="Helsinki",
              energyKwh=320, renewablePct=88)
    result = score_one(s)
    assert result["greenScore"] == 80
    assert result["tier"] == "A"

def test_low_renewable_site_is_warned():
    s = Site(siteId="s2", name="Warsaw Plant", city="Warsaw",
              energyKwh=880, renewablePct=24)
    result = score_one(s)
    assert result["tier"] == "C"
    assert "renewables" in result["tip"].lower()
```

Because GreenGrid hosts this service itself, it ships as a small container. The workload needs the service URL and the Entra configuration for its delegated API scope, not the scoring source code.

This container is intentionally minimal. Production also needs a non-root runtime, image scanning, health probes, patching, and deployment policy.

```
# Dockerfile - GreenGrid packages and hosts its own scoring service.
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY greengrid_saas.py auth.py ./
EXPOSE 8787
CMD ["uvicorn", "greengrid_saas:app", "--host", "0.0.0.0", "--port", "8787"]
```

9.3 Milestone 1: the workload calls the service

With the algorithm running, the workload is the join. Milestone 1 puts a working scorecard on screen using seed data for the sites but a real call to the SaaS for the scores, the shapes that cross the boundary, a client with one responsibility, and a screen that takes a `getSites` function rather than a fixed source:

The frontend shares the service contract and acquires a token for the GreenGrid API:

```
import { WorkloadClientAPI } from "@ms-fabric/workload-client";

// contracts.ts - the shapes shared with the Python service.
export type SiteRecord = { siteId: string; name: string; city: string; energyKwh: number; renewablePct: number; };
export type ScoredSite = SiteRecord & { greenScore: number; tier: "A" | "B" | "C"; tip: string; };
export type ScoreResponse = { sites: ScoredSite[]; summary: { totalSites: number; avgScore: number } };

// greengridClient.ts - acquire a delegated token for the GreenGrid API.
// GREEN_GRID_API_SCOPES comes from the GreenGrid Entra app registration.
export async function scorePortfolio(
  client: WorkloadClientAPI,
  sites: SiteRecord[],
): Promise<ScoreResponse> {
  const token = await client.auth.acquireFrontendAccessToken({
    scopes: GREEN_GRID_API_SCOPES,
  });
  const res = await fetch(`${SAAS_URL}/score`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ sites }),
  });
  if (!res.ok) throw new Error(`GreenGrid SaaS error ${res.status}`);
  return res.json();
}
```

The scorecard keeps data loading behind `getSites` so the source can change later:

```
// Scorecard.tsx - driven by getSites, so the data source can change later.
export function Scorecard({
  client,
  getSites,
}): {
  client: WorkloadClientAPI;
  getSites: () => Promise<SiteRecord[]>;
} {
  const [data, setData] = useState<ScoreResponse | null>(null);
  const [error, setError] = useState<string | null>(null);
  useEffect(() => {
    getSites()
      .then((sites) => scorePortfolio(client, sites))
      .then(setData)
      .catch((e) => setError(String(e)));
  }, [client, getSites]);
  if (error) return <MessageBar intent="error">Failed: {error}</MessageBar>;
  if (!data) return <Spinner label="Scoring sites..." />;
  return <PortfolioView data={data} />; // average score, then a card per site
}
```

The default view renders the screen with seed data, while the scores come from an authenticated call to GreenGrid's service. Stopping that service makes the screen report the failure. The browser holds a short-lived delegated token, not a publisher API key.

9.4 Milestones 2 and 3: real data and a native screen

The seed array stood in for customer data stored in a Lakehouse CSV at `Files/sites.csv`. The frontend requests a separate OneLake Storage token to read that file. Because the screen depends on `getSites`, the component and scoring client do not change.

The first milestone binds the scorecard to seed data:

```
<Scorecard
  client={client}
  getSites={() => Promise.resolve(seedSites)}
/>
```

The next milestone replaces only the data function:

```
<Scorecard
  client={client}
  getSites={() =>
    readSitesCsv(client, workspaceId, lakehouseId).then(parseCsv)
  }
/>
```

The third milestone adds Fluent UI components and follows the theme supplied by Fabric. GreenGrid stops there: no remote workload jobs, no lifecycle endpoint, and no production release pipeline. The scoring SaaS remains a real external data boundary that a production publisher would document and operate.

The same scored portfolio reads differently as a map. Figure 9.2 is a second view of the item, the five sites placed geographically and colored by tier, with the Tier C plant in Warsaw standing out in red against the greener sites to the west.

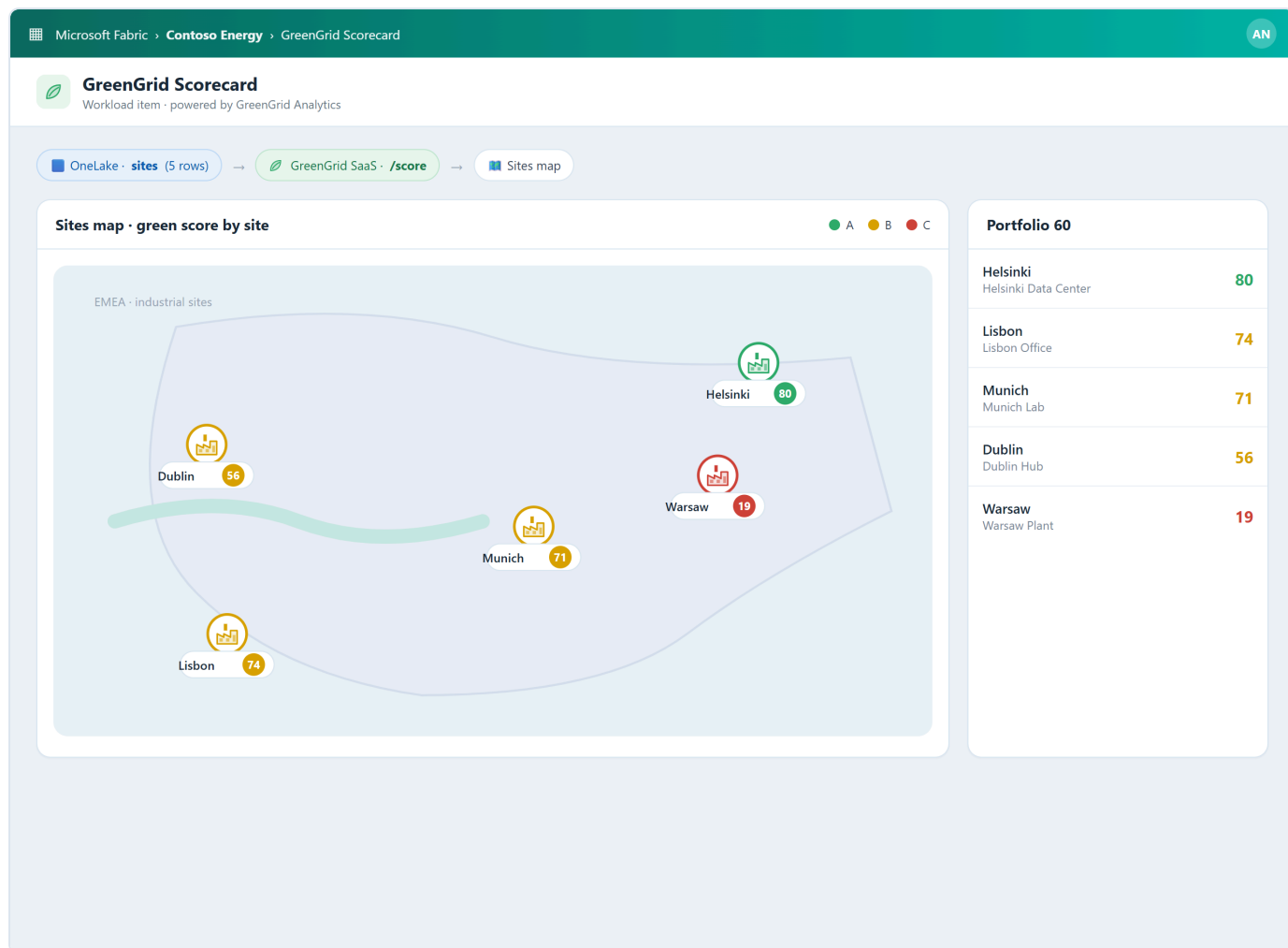


Figure 9.2 · The third milestone, the scorecard turned graphical. The data behind the markers is identical to Figure 9.1, read from OneLake and scored by the same service. Only the rendering changed, which is exactly what keeping the data behind a function buys you.

Go to production

Key takeaways

- Some development conveniences become production configuration, but authorization, isolation, migration, support, and rollback require design and evidence.
- Host the frontend under a verified domain and separate user delegation, backend application credentials, and managed identities.
- Treat data sent to publisher-hosted code as a disclosed external boundary. Custom items do not receive sensitivity protection automatically.
- Build the package with local schema checks, then use the separate publishing validator at the correct stage.

10. From developer mode to production

10.1 What changes

The Dev Gateway is a development tool. In production, Fabric loads the frontend directly from the HTTPS endpoint declared in the manifest. The code changes less than the environment around it: localhost becomes a verified publisher domain, development credentials become production identity flows, console output becomes monitored telemetry, and a personal dev instance becomes an uploaded and assigned workload or a cross-tenant publication.

Concern	Developer mode	Production
Delivery to Fabric	Dev Gateway bridges localhost	Fabric loads your hosted URL directly
Frontend host	localhost (Dev Server)	Your cloud, HTTPS, on a verified domain
Backend identity	Development app and local credentials	Backend app credential for remote flows, managed identity where supported
Telemetry	Console and local logs	Application Insights, correlation IDs retained
Reaching users	You, in your dev workspace	Upload, activate, assign, or publish across tenants

The table captures configuration substitutions. It does not cover the design and operational work added by production:

Change type	Examples	Exit evidence
Configuration	URLs, domains, CORS, CSP, app IDs, environment values	Reviewed configuration and successful deployment
Design	Authorization, tenant isolation, data retention, definition migration, permission evolution	Architecture decision, threat model, compatibility tests
Operations	Monitoring, support, incident response, release recovery, capacity and service ownership	Dashboards, alerts, runbooks, support contacts, recovery exercise

Domain and hosting come first because application registrations and manifest endpoints depend on them. Identity comes next: configure delegated frontend scopes, the backend application and its rotated credential where remote OBO is used, and managed identity for supported service calls. Telemetry and packaging wrap a workload that already runs.

10.2 What to verify across the transition

A handful of things change together. The frontend uses HTTPS under the verified domain. Its framing policy must allow the supported Fabric and Power BI portal families. The backend permits the frontend origin through CORS. Each token must match its target resource, audience, and scopes. Every package upload uses a new version. The frontend acquires fresh tokens through the SDK rather than caching them past expiry.

Two header settings often decide whether the product works inside Fabric. The frontend host must allow portal framing:

```
Content-Security-Policy: frame-ancestors https://*.fabric.microsoft.com https://*.powerbi.com;
```

A backend called by that frontend must allow only the required origin and methods:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://your-frontend.example.com"], # your verified frontend
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)
```

Treat the header as an example, not a permanent allow-list. Check the current publishing requirements and test item creation under both portal domain families before release.

| 11. Hosting, domain, and identity

11.1 Hosting, the verified domain, and the resource ID

A workload's frontend is served from the publisher's cloud over HTTPS and loaded by Fabric in the iframe. A backend is optional. Public publishing requires a verified custom domain, and an *.onmicrosoft.com subdomain is not allowed. The resource ID ties the frontend, backend, and identity together:

```
https://<verified-domain>/<frontend>/<backend>/<workload-id>/<optional>
```

The frontend and backend URLs sit below the verified domain, the reply URL matches the frontend host, the redirect URI ends in /close, and every endpoint uses HTTPS. For a fictional workload under contoso.com:

```
Resource ID: https://datafactory.contoso.com/feserver/beserver/Contoso.SalesInsights/1
Frontend:    https://feserver.datafactory.contoso.com
Backend:     https://beserver.datafactory.contoso.com/workload
Redirect URI: https://feserver.datafactory.contoso.com/close
```

Each URL is a subdomain of the verified domain, the reply URL matches the frontend host, and every endpoint uses HTTPS. That is the pattern a production workload follows.

11.2 User delegation, backend credentials, and managed identities

Production identity is a set of flows, not one credential. The frontend requests delegated resource tokens for the signed-in user. The current remote-hosting reference uses a backend Entra application credential for OBO and service-to-service workload control. That credential stays on the server, belongs in Key Vault or an equivalent secret store, and must be rotated. Public distribution also requires a multitenant setup and verified-publisher prerequisites.

Managed identity remains useful for supported Azure and Fabric API calls made as the publisher service. It does not replace every backend application credential in the remote workload flow. DefaultAzureCredential supports developer identity locally and managed identity in Azure:

```
# backend_identity.py - service access where managed identity is supported.
from azure.identity import DefaultAzureCredential

# Locally: your az login / VS Code identity. In Azure: the managed identity.
_credential = DefaultAzureCredential()

def fabric_token() -> str:
    # ".default" asks for the app's configured permissions; the platform rotates it.
    token = _credential.get_token("https://api.fabric.microsoft.com/.default")
    return token.token

def call_fabric(path: str) -> dict:
    import requests
    resp = requests.get(
        f"https://api.fabric.microsoft.com/v1/{path}",
        headers={"Authorization": f"Bearer {fabric_token()}"}, timeout=30,
    )
    resp.raise_for_status()
    return resp.json()
```

Decide whose authority each call needs. A call made for the user uses delegation and, when the backend needs another resource, OBO. A publisher operation may use the backend application or a managed identity where the target API supports it. Document application permissions separately from delegated permissions and do not use a broad service identity as a shortcut around user authorization.

12. Security and compliance

12.1 Data boundaries, labels, and publisher responsibility

A workload can read Fabric data in place, but that does not mean all data stays in the tenant. Microsoft warns administrators that workload interactions can send user data and access tokens, including identity information, to the publisher. Any call from the iframe to a publisher API creates that boundary. The publisher must document what crosses it, where processing occurs, how long data is retained, how tenants are isolated, and which subprocessors receive it.

Stable concept: Fabric owns the platform boundary. The publisher owns what crosses it.

Risk or boundary	Expected control	Responsible party	Evidence before release
Cross-tenant data mix	Tenant-aware keys, authorization, storage, cache, and tests	Publisher	Isolation test results and architecture record
User access to workspace/item data	Resource authorization after token validation	Fabric and publisher	Permission matrix and negative access tests
Data sent to publisher	Minimized payload, documented purpose, residency, retention, and subprocessors	Publisher	Data-flow inventory, privacy material, attestation
Logs and telemetry	No tokens or customer payloads, controlled retention and access	Publisher	Logging schema, sample records, retention policy
Backend credentials	Server-side storage, least privilege, rotation, alerting	Publisher	Secret-store policy and rotation evidence
New permissions	Versioned consent change with customer communication	Publisher and customer admin	Permission diff, release notes, consent test
Platform incident escalation	Correlation identifiers propagated where available	Fabric and publisher	Trace showing ActivityId / RequestId propagation

Microsoft's [additional-workloads warning](#) states that sensitivity labels and protection settings, including encryption, are not applied to items created with workloads. The workload still has to respect controls on source data and prevent unauthorized export. It cannot claim that the custom item inherits encryption or DLP behavior without implementing and validating that behavior. The publishing attestation and privacy material must describe what the product provides.

Design for the smallest useful transfer. If a scoring service needs site identifier, energy use, and renewable percentage, send those fields rather than the whole table. Do not write customer rows to logs, shared caches, or analytics stores unless the product contract explicitly requires it and the customer has accepted the residency, retention, and access model.

Current public publishing requirements also include security and privacy assessments, Conditional Access support, SDK-based Entra token acquisition, no third-party cookies, and only essential same-origin HTTP-only cookies after authentication. These are release criteria, not optional polish.

12.2 Secrets, telemetry, monitoring, and support

Nothing sensitive belongs in the frontend bundle or a URL. Store backend credentials in Key Vault or an equivalent server-side store and rotate them. Use managed identity where the target supports it:

```
# secrets.py - fetch a rotated secret from Key Vault with the managed identity.
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

_client = SecretClient(
    vault_url="https://your-vault.vault.azure.net",
    credential=DefaultAzureCredential(),
)

def backend_client_secret() -> str:
    return _client.get_secret("workload-backend-client-secret").value
```

Where Fabric provides ActivityId or RequestId, carry those values into publisher logs. Application Insights can capture the service telemetry:

```
# telemetry.py - send traces and metrics to Application Insights.
from azure.monitor.opentelemetry import configure_azure_monitor
import logging
import os

configure_azure_monitor(
    connection_string=os.environ["APPLICATIONINSIGHTS_CONNECTION_STRING"]
)
logger = logging.getLogger("workload")

def record_scoring(site_count: int, activity_id: str | None) -> None:
    logger.info(
        "scored portfolio",
        extra={"site_count": site_count, "activity_id": activity_id},
    )
```

Use the identifiers available on a request to correlate publisher telemetry with Fabric activity. Alert on failed requests, job failures, and authentication errors without logging tokens or customer payloads.

Support remains the publisher's responsibility. Public publishing requires working help and support links, documented contact methods and service expectations, and a livesite contact at GA. Microsoft can investigate platform requests when you provide the relevant request identifiers. It does not operate the publisher's SaaS or customer support process.

13. Packaging, validation, and CI/CD

13.1 Package build, schema checks, and publishing validation

A workload is delivered as a .nupkg, the NuGet archive format reused for the Fabric manifest package. It contains declarations, assets, and locale files, not the running frontend or backend. Every upload needs a unique version.

Treat a release as a compatibility set, even though its components are deployed through different channels:

Component	Version source	Delivery	Compatibility evidence
Manifest package	Release tag and WorkloadManifest.xml version	.nupkg upload	Version match, XML/XSD checks, package hash
Hosted frontend	Immutable artifact or image digest	Frontend hosting platform	Item smoke tests against target package/backend

Component	Version source	Delivery	Compatibility evidence
Hosted backend	Immutable artifact or image digest	Backend hosting platform	API/OBO contract tests and health evidence
Item-definition schema	schemaVersion in definition parts	Read/write through item APIs	Migration and backward-compatibility tests
Permission set	Entra configuration plus package requirements	Admin consent	Permission diff and consent-path test

Changing only the package does not roll back a separately deployed frontend or backend. Record all component versions in the release evidence and define which combinations are supported.

Build the package with local XML and XSD checks:

```
pwsh ./scripts/Build/BuildManifestPackage.ps1 `
  -Environment prod `
  -ValidateFiles $true
```

Verify that the .nupkg exists, opens, and contains the expected version because the pinned script does not check every native pack exit code.

After publishing the workload to a tenant, run the separate validator:

```
cd fabric-extensibility-toolkit-validator/validator
npm install
node index.js --workload-name "Contoso.MyWorkload" --workload-stage "Preview"
```

Appendix B records the validator version, prerequisites, options, and output. A successful self-validation is evidence for the publishing review, not Microsoft approval.

Use the release tag as the version source and fail the build when the manifest differs:

```
# verify_version.py - fail the build if the release tag and manifest disagree.
import os
import pathlib
import re

manifest = pathlib.Path("Workload/Manifest/WorkloadManifest.xml")
text = manifest.read_text(encoding="utf-8")
manifest_version = re.search(r"<Version>([^\<]+)</Version>", text).group(1)
release_version = os.environ["RELEASE_VERSION"].removeprefix("v")

if manifest_version != release_version:
    raise SystemExit(
        f"Manifest version {manifest_version} does not match release {release_version}"
    )

print(f"verified {release_version}")
```

13.2 Automating the pipeline

The pipeline builds the hosted components and package as one compatibility set. This Bicep excerpt shows the hosting shape, not a complete landing zone:

```
// hosting.bicep - publisher hosting, with managed identity for supported service access.
param location string = resourceGroup().location

resource identity 'Microsoft.ManagedIdentity/userAssignedIdentities@2023-01-31' = {
  name: 'id-workload-backend'
  location: location
}

resource plan 'Microsoft.Web/serverfarms@2023-12-01' = {
  name: 'plan-workload'
  location: location
  sku: { name: 'P1v3' }
}

resource backend 'Microsoft.Web/sites@2023-12-01' = {
  name: 'beserver-yourworkload'
  location: location
  identity: { type: 'UserAssigned', userAssignedIdentities: { '${identity.id}': {} } }
  properties: { serverFarmId: plan.id, httpsOnly: true }
}
```

The toolkit scripts are PowerShell-based, so this skeleton uses a Windows runner. A production repository also needs a committed lockfile, environment approvals, artifact integrity, and rollback controls:

```
# .github/workflows/release.yml
name: release-workload
on:
  push:
    tags: ["v*"]
jobs:
  build:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20" }
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - name: Build frontend
        run: npm ci && npm run build:prod
        working-directory: Workload
      - name: Verify release and manifest versions
        run: python build/verify_version.py
        env:
          RELEASE_VERSION: ${ github.ref_name }
      - name: Build package and validate XML/XSD
        shell: pwsh
        run: >
          ./scripts/Build/BuildManifestPackage.ps1
          -Environment prod
          -ValidateFiles $true
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: workload-package
          path: "**/*.nupkg"
```

Current public documentation keeps package upload in the Admin Portal. Fabric Admin Workloads APIs can list published workloads and automate assignment to a tenant, capacity, or workspace, but they do not expose a package-upload operation. The documented [list workloads](#), [list assignments](#), [create assignment](#), and [delete assignment](#) operations each state a maximum of 200 requests per hour and `PrivateLinksUnsupported`. The documentation does not define whether the limit is partitioned by caller, tenant, service principal, or operation.

Deploy the frontend and backend with the supported mechanism for the chosen hosting platform. If the pipeline targets Azure, OpenID Connect through `azure/login` avoids a long-lived pipeline credential. Keep package upload as an explicit release step until Microsoft documents an upload API, then automate activation and assignment separately where it helps.

Validation stage	Input	Output	Responsible party	Release evidence
Build and local schema checks	Source, release version, manifest package	Frontend/backend artifacts and .nupkg	Engineering pipeline	Logs, hashes, version match
Item behavior	Deployed services plus development or test package	Verified create/open/save/error flows	Engineering and QA	Test run with item/workspace IDs
Publishing validator	Workload published to the target stage	Automated and manual findings	Publisher	Validator report and issue disposition
Stage review and activation	Package, evidence, support/compliance material	Selected, Preview, or GA availability	Publisher and Fabric administrators	Approval, activation, assignment, consent record

14. Patterns and anti-patterns

14.1 Patterns that hold up

Decision	Pattern	Consequence
Let the data source evolve without rewriting the view	Put data access behind a typed function	Seed, OneLake, and publisher sources can change behind one UI contract
Keep item state reviewable and recoverable	Store compact configuration in the item definition and business data in the data plane	Git, migration, and data recovery have separate contracts
Preserve identity boundaries	Request a token for the called resource, use OBO only for a downstream resource, and use managed identity only where supported	A bearer cannot drift between audiences or replace item authorization
Claim only configured integrations	Validate catalog, jobs, monitoring, Git, deployment, labels, and remote operations separately	Product documentation matches the behavior customers can observe
Make failures operable	Provide loading, empty, and error states, then propagate available correlation identifiers	Support can distinguish UI state, transport, identity, and service failures
Limit the publisher boundary	Transfer only required fields and document publisher processing	Security, residency, retention, and support ownership stays explicit

14.2 Anti-patterns to avoid

Anti-pattern	Failure it creates
Secret in the frontend or URL	Any user or intermediary that can inspect the request can recover it
Token sent to the wrong audience	The target rejects it, or an API accepts a bearer it was not designed to trust
Service identity used instead of user and item authorization	The backend can cross the caller's intended resource boundary
Customer payload written to shared logs or caches	Tenant isolation and retention become harder to prove
Built-in integration claimed without configuration and a test	Product documentation promises behavior the item might not provide
Local schema check treated as publishing approval	A technically valid package reaches review without satisfying the target stage
Package version reused	The upload is rejected and the release cannot be traced cleanly
Existing item assumed to match the latest definition schema	Older items fail to open or lose configuration during migration

Distribute

Key takeaways

- Upload through the Admin Portal, then activate and assign the workload at tenant, capacity, or workspace scope.
- Workload Hub handles Fabric discovery, consent, and assignment. Microsoft Marketplace provides the required public commercial listing.
- The naming form follows the audience: `Org.[Name]` is internal, while `[Publisher].[Workload]` supports cross-tenant publication.
- A published workload has a life: updates, item migration, deprecation, and cross-tenant consent.
- Selected-tenant testing, Preview, and GA are distinct stages with different requirements.

15. Make it available in your tenant

15.1 Admin Portal upload, activation, and assignment

Package upload happens in the Admin Portal on the Workloads page. The *Publish* tab accepts a new `.nupkg` version. *Manage my tenant* is for workloads the organization can activate, consent to, and assign. Upload alone does not expose the item type to every workspace.

After upload, activate the version and assign the workload at the required scope: tenant, capacity, or workspace. Current Fabric Admin Workloads APIs can list published workloads and automate assignment, but the public API does not upload the package itself. An active version must be deactivated before it can be deleted.

15.2 Internal publishing with `Org.[Name]`

An `Org.[Name]` workload needs no separate workload-name registration and is intended for the publishing tenant. General hosting, identity, manifest, security, and tenant requirements still apply. Assign it only where it is needed rather than enabling it across the organization by default.

While Dev Gateway is connected, a local workload can take precedence over the uploaded version for the configured workspaces. Check the gateway before diagnosing an apparent version rollback.

16. Publish across tenants: Workload Hub and Microsoft Marketplace

16.1 Selected tenants, Preview, and GA

Cross-tenant publication uses a `[Publisher].[Workload]` name. The name and publishing tenant are reserved permanently when the publisher confirms the first upload.

Cross-tenant rollout moves from selected tenants to Preview and GA. The selected-tenant stage currently accepts up to twenty tenant IDs and can take up to ten minutes to propagate.

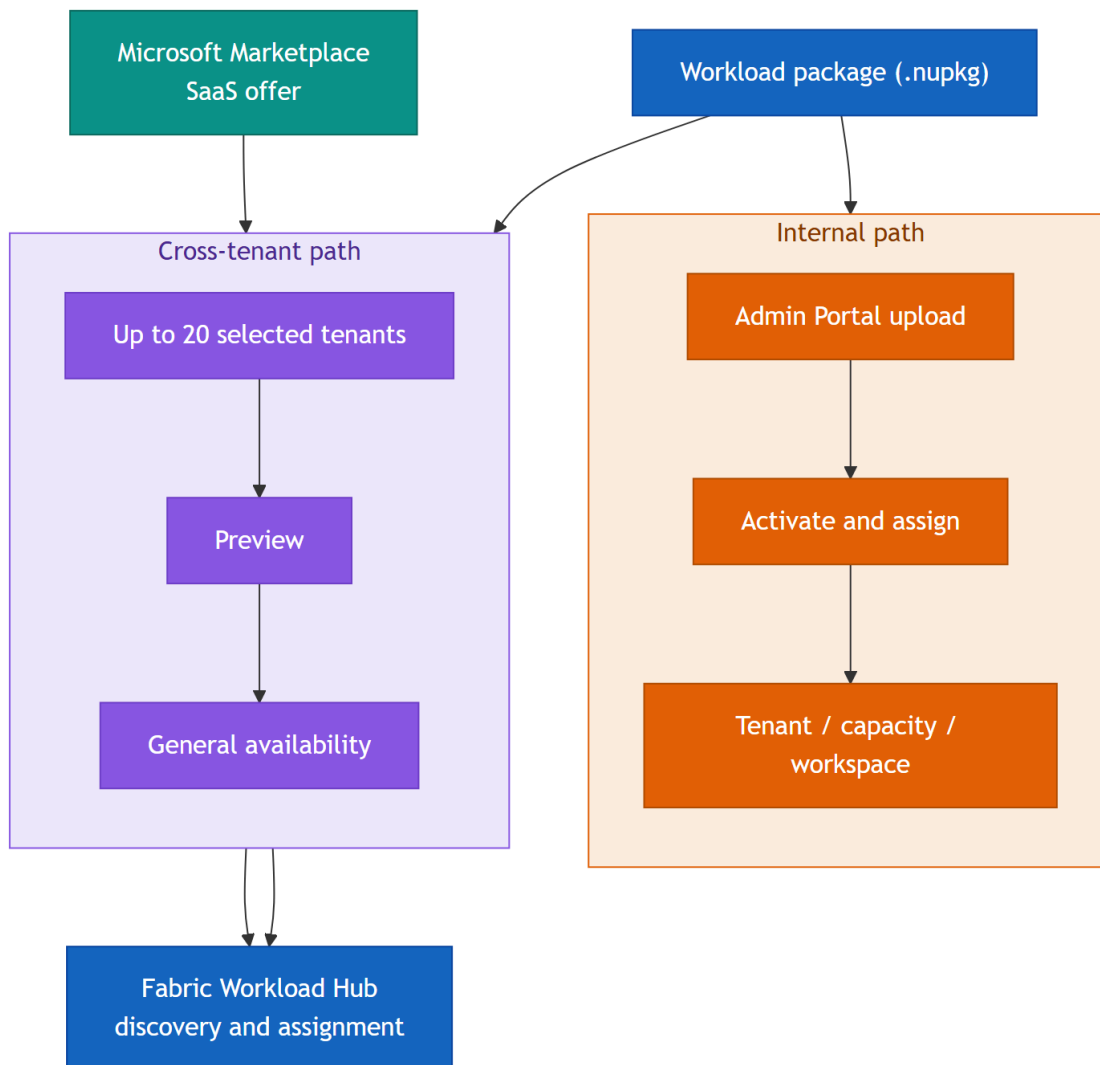
Stage	Audience	Publisher action	Customer or administrator action	Result
Internal	Publishing tenant	Upload and activate <code>Org.[Name]</code>	Assign it to the intended scope	The item is available inside the organization
Selected tenants	Up to 20 nominated tenants	Register the publisher name and nominate pilot tenants	Enable unvalidated workloads and run the pilot	The workload appears in the selected tenants
Preview	All Fabric tenants, with a Preview indication	Submit the publishing request and evidence	Consent and assign after approval	The workload is publicly discoverable in Preview
GA	All Fabric tenants	Submit GA evidence and operating readiness	Consent, assign, and use the supported product	The Preview indication is removed

Appendix F keeps the exact stage sources and current discrepancies.

The actions are separate:

- **Publish** registers a package version in the publishing tenant.
- **Activate** makes a published version eligible for use.

- **Consent** records the customer administrator's acceptance of permissions and workload access.
- **Assign** makes the workload available at the selected tenant, capacity, or workspace scope.



Selected tenants let the publisher test installation, consent, onboarding, support, and real customer boundaries before public distribution.

16.2 Publishing requirements, commerce, and support

Fabric Workload Hub and Microsoft Marketplace serve different roles. Workload Hub is where Fabric administrators discover, consent to, and assign the workload. Preview and GA currently require a Microsoft Marketplace SaaS offer in Partner Center, including nontransactable offers.

The current [SaaS listing options](#) are **Contact me**, **Free trial**, **Get it now (Free)**, and **Sell through Microsoft**. Only the transactable option uses Microsoft-facilitated billing. Existing licensing or publisher-managed sales can use a nontransactable listing, but the Marketplace offer remains part of the public Fabric publishing requirements.

Publishing is one step in the customer journey. A prospect still needs to understand the business outcome, obtain administrative approval, configure the service, and reach a useful result. Design that journey as carefully as the item editor. For an existing SaaS customer, explain how the workload connects to the current subscription and what additional setup is required. For a new customer, make evaluation, purchasing, entitlement, and support clear. The Marketplace offer and the Fabric workload should make the same product promise even when commercial and administrative actions happen in different places.

Microsoft co-sell can complement that route, but it has separate [readiness and eligibility requirements](#). A live listing creates an opportunity for distribution. Selling still depends on a clear use case, credible evidence, and follow-through with customers.

Preview and GA also require publishing evidence, support and privacy material, publisher verification, and the active stage requirements. Appendix F records the current trial and requirement conflicts. Appendix D keeps the fulfillment webhook example.

16.3 Choosing a path

First apply the product-fit criteria from section 1.3. If a workload is justified, choose distribution according to audience and operating commitment:

Path	Choose it when	Durable cost	Hard-to-change decision
Internal Org. [Name]	One organization needs the item and public discovery adds no value	Tenant administration, compatibility, support	Later migration to a publisher identity
Selected tenants	The publisher needs real customer validation before a public program	Multi-tenant operations, consent, support, customer onboarding	Publisher name and publishing tenant
Preview	Product, support, security evidence, and Marketplace setup are ready for a broader audience	Review findings, Marketplace operations, public support	Public product and permission contract
GA	The service can sustain production customers and livesite obligations	Ongoing compatibility, support, commerce, monitoring, incident response	Long-lived item types, migrations, and deprecation policy

The channel can widen the addressable audience, but it does not guarantee adoption or sales. Confirm the permanent publisher name and tenant only after the product and operating model can support the chosen path.

Start with a small group of selected-tenant pilots. Continue only when customers reach a useful result, onboarding effort is repeatable, the workload sees regular use, and the path from pilot to paid service is clear enough to support.

17. The post-publish lifecycle

17.1 Updates, migration, and deprecation

Every release uses a new package version. If the definition shape changes, the frontend must still open older items and upgrade them safely. That compatibility is part of customer trust:

```
# migrate.py - bring an item definition forward across versions on load.
def migrate(definition: dict) -> dict:
    version = definition.get("schemaVersion", 1)
    if version < 2:
        definition["source"] = definition.get("source", "seed") # field added in v2
        definition["schemaVersion"] = 2
    if version < 3:
        definition["horizonDays"] = definition.get("horizonDays", 14) # added in v3
        definition["schemaVersion"] = 3
    return definition
```

Retiring a version or item type is a managed step. Existing items still need to open or migrate, and an active workload version must be deactivated before deletion. Keep customer data separate from the definition so a frontend release does not become the only way to recover it.

17.2 Monitoring, consent, rollback, and feature flags

Telemetry can measure adoption and failures without collecting customer payloads. A release that adds permissions changes the tenant consent contract, so explain the change before administrators are asked to approve it.

Fabric requires a package version it has not seen before, so rollback means publishing known-good behavior under a new forward version. Keep previous code and manifests, use feature flags where they reduce risk, and test the recovery path before customers need it.

18. Recap and next steps

A workload gives a publisher-owned product a native item and experience inside Fabric. The package, hosted application, identity flows, and data plane remain separate contracts that must evolve together.

The business decision is just as concrete: choose a customer segment, a recurring problem, a pilot, and the signal that will prove value. An ISV may extend an existing product. An SDC may turn repeated project work into a supported accelerator. Neither path becomes a product without ownership, support, and a credible route from evaluation to ongoing use.

Continue with the action that matches your role:

Role	Next action	Use
Product owner / ISV or SDC lead	Name the target customer, problem, pilot, value signal, and intended commercial path	Sections 1 and 16
Architect / security lead	Approve the boundary, identity, data-transfer, and evidence models	Sections 2, 4, 11, and 12
Developer	Follow the Fast track and record the first successful and failed item flows	Sections 0, 2, 5, 6, 8, and 9
Release engineer	Define the release version source, component compatibility set, and validation evidence	Sections 10, 13, and 17
Publisher / operations owner	Prepare assignment, consent, support, Marketplace, monitoring, and recovery processes	Sections 15 through 17 and Appendices E through G

| Appendices

Appendix A: Manifest package reference

File	Scope	Review before release
WorkloadManifest.xml	Workload identity, version, hosting mode, Entra applications, service endpoints	Workload name, schema, app IDs, URLs, version
Product.json	Product presentation, support, privacy, terms, certification, license links	Working URLs, publisher metadata, localization
{Item}.xml	Platform definition for one item type	Item name, capabilities, jobs, lifecycle, catalog integration
{Item}.json	Frontend presentation and behavior for the same item type	Editor route, create experience, settings, operations

Source-to-package map:

Source input	Built package location	Packaging note
WorkloadManifest.xml	BE/WorkloadManifest.xml	Workload-level XML
items/<Item>/<Item>.xml	BE/<Item>.xml	Selected item XML files are flattened
Product.json	FE/Product.json	Product and support metadata
items/<Item>/<Item>.json	FE/<Item>.json	Selected item JSON files are flattened
assets/images, assets/locales	FE/assets/...	Only packaged assets and locale material
XSDs and ManifestPackage.nuspec	Build input and NuGet metadata	Not hosted application code
Frontend bundle, backend code, environment files, business data, item-instance definitions	Not included	Deploy or store through their own runtime and data-plane paths

Current package limits:

Package limit	Maximum
Item types	10
Package size	20 MB

Package limit	Maximum
Packaged assets	15
Size per asset	1.5 MB
Product.json	50 KB

Other current thresholds:

Area	Current value	Qualification and source
Cross-tenant workload name	Workload portion at most 32 characters	Publishing naming rule . The same source does not establish a separate Org.* length limit
Item manifest filenames	At most 32 English alphanumeric or hyphen characters, unique in the package	Manifest package limits
OneLake Catalog categories	Maximum two per item	Catalog configuration
Item-definition files	Guidance says a reasonable number, up to five	Definition guidance . Not documented as a hard API threshold
Publisher hosting	Publishing text requires a 99.9 percent uptime SLA and page load under three seconds	General requirements . Measurement method is not specified
Lifecycle response	Thirty seconds recommended	Lifecycle endpoint guidance . Not stated as a hard timeout

Use Org.[Name] for the publishing tenant. Use [Publisher].[Workload] for selected tenants, Preview, and GA. The publisher name and publishing tenant become permanent when the first publisher-named upload is confirmed.

Capability matrix:

Capability	Mechanism or declaration	Publisher work	Conditions and limits	Source	Implementation section
Workspace item and permissions	Workload/item manifests plus Fabric item APIs	Define the item type and test authorization	Does not imply every built-in integration	Architecture	2.2, 6.1
OneLake Catalog	oneLakeCatalogCategory and separate supportedInDatahubL1 flag	Select accurate categories and test both discovery surfaces	At least one category is required for catalog appearance. Maximum two categories	Catalog configuration	2.2, 6.3
Item definitions, Git, deployment	Human-readable definition parts	Define schema, serialization, compatibility, and migrations	Guidance says a reasonable number of files, up to five. It is not documented as an enforced API quota	Definition structure and Git	2.2, 6.1, 6.2, 17
Jobs	<JobScheduler> plus publisher remote endpoint	Execute jobs and implement the selected status, cancel, retry, or detail paths	Fabric schedules and monitors. Execution stays with the publisher. Deduplication settings are not a numeric concurrency quota	Remote jobs	6.3
Monitoring	Configured jobs plus optional supportedInMonitoringHub, itemJobActionConfig, handlers, and Recent Runs settings	Verify base listing, filters, actions, and Recent Runs separately	Configured jobs appear in Monitoring Hub. The extra integrations require their own declarations and handlers	Monitoring and actions, Recent Runs	6.3
Item lifecycle	Remote lifecycle declarations and endpoint	Handle create/update, soft delete, hard delete, and restore	Delete may lack a subject token and cannot be blocked. Enabling OnDelete also requires restore handling	Lifecycle behavior, restore requirement	6.3, 17
Remote backend/endpoints	Remote hosting configuration and Entra backend app	Implement SubjectAndAppToken1.0, operation authorization, availability, and support	Lifecycle REST contracts are published. Generic remote-endpoint material and official job-route samples remain inconsistent, so validate the target stage	Remote authentication, incomplete endpoint specification	2.1, 4, 11
Sensitivity labels and protection	Source-data policy plus publisher controls and attestation	Prevent unauthorized transfer and document the behavior the product provides	Microsoft states that sensitivity labels and protection settings, including encryption, are not applied to items created with workloads	Additional-workloads warning	12

Appendix B: Setup, development, package, and validator commands

The command reference below uses the pinned toolkit and validator versions:

Task	Actor and prerequisite	Start directory	Command or action	Parameters to adapt	Expected output	Versioned source and status
Direct setup	Developer with Entra app and development-workspace rights	scripts/Setup	<code>pwsh .\SetupWorkload.ps1 -WorkloadName "Org.YourWorkload"</code>	WorkloadName, optional display name, frontend/backend app IDs, development workspace, Boolean Force, workload version	Environment and app configuration, then a package-build attempt	Signature . Source-inspected only. The final handoff passes an unsupported <code>-Force</code>
Compatibility setup	Same as direct setup	Repository root, following Microsoft Learn	<code>pwsh .\scripts/Setup/Setup.ps1</code>	Forwarded setup parameters	Same intended result as direct setup	Wrapper , Learn command . Same pinned build warning
Create item type	Developer after setup	scripts/Setup	<code>pwsh .\CreateNewItem.ps1 -ItemName "Forecast" -srcItemName "HelloWorld"</code>	Required ItemName, optional srcItemName defaulting to HelloWorld	Copied item files, followed by manual wiring	Signature , follow-up
Serve local frontend	Developer with generated environment files	scripts/Run or repository root with the root-relative path	<code>pwsh .\StartDevServer.ps1</code>	No parameters	Local SPA, assets, and development manifest endpoints	Signature , documented invocations . Source-inspected, not run
Register local workload	Developer with personal Fabric Developer Mode	scripts/Run or repository root with the root-relative path, second terminal	<code>pwsh .\StartDevGateway.ps1</code>	Boolean InteractiveLogin, default <code>\$true</code> . On Linux use <code>-InteractiveLogin \$false</code>	Local workload registration pointing Fabric to Dev Server	Signature , platform branches . It always rebuilds the dev manifest
Switch to remote hosting	Developer preparing remote endpoints	scripts/Setup or repository root with the root-relative path	<code>pwsh .\SwitchToRemoteHosting.ps1 ...</code>	WorkloadRoot, BackendAppId, BackendAudience, BackendUrl, TenantId, EnableOneLakeLogging, BackendClientSecret, switch Force	Remote hosting manifest, app, and environment changes	Signature and examples . At this pin it updates <code>.env.dev</code> and <code>.env.test</code> , writes the backend secret to those local files, and migrates only <code>HelloWorldItem.xml</code> . Review and complete the changes before use
Build manifest package	Release engineer	Repository root or any directory when using the root-relative path	<code>pwsh .\scripts/Build/BuildManifestPackage.ps1 -Environment prod -ValidateFiles \$true</code>	Boolean ValidateFiles, default <code>\$false</code> . Environment, default dev, with no ValidateSet. Use a matching environment file	Versioned <code>.nupkg</code> plus local XML/XSD findings	Signature , build logic . No Force parameter. Verify the artifact because native pack exit codes are not checked
Validate publication	Publisher with Node.js 20 or later, Chrome/Chromium, Workload Hub access, and a workload published in the signed-in tenant	<code>fabric-extensibility-toolkit-validator/validator</code>	<code>npm install, then node .\index.js -workload-name "Contoso.MyWorkload" --workload-stage "Preview"</code>	Required workload name. Canonical stage Preview or GeneralAvailability. Optional <code>--print-testcases</code> , <code>--skipInteractiveTests</code> , <code>--skipPassedTests</code> , <code>--skipUpdates</code>	Automated and guided manual tests plus result files under <code>Results\<workload>\<stage>\<validation-id>\. Not Microsoft approval</code>	Validator tag <code>v2025.12.1</code> , commit <code>78e17c3...</code> , CLI . Source-inspected, not run

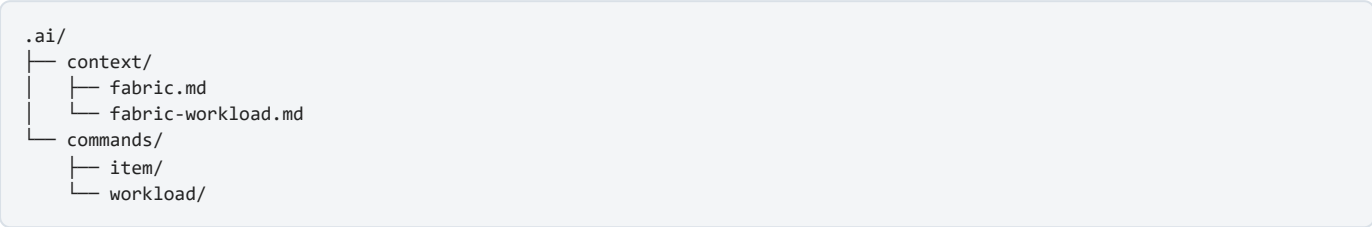
Package upload remains an Admin Portal action in current public documentation. Fabric Admin Workloads APIs automate listing and assignment of an already published workload, not package upload.

Appendix C: AI guidance reference

Source	Purpose	Authority level
Microsoft Learn	Current platform behavior and publishing requirements	Product authority
Repository scripts and installed SDK types	Commands and APIs present in the checked-out version	Executable truth for that version

Source	Purpose	Authority level
.ai/context and .ai/commands	Repository knowledge and task procedures	Useful guidance, may become stale
copilot-instructions.md, scoped instructions, @fabric	Copilot-specific routing and conventions	Host-specific guidance
Community UX MCP referenced by the toolkit	Optional retrieval over UX material	Community dependency, review before use

Repository guidance map:



The optional community UX MCP needs a locally reviewed command and dependency set before it is enabled:

```

{
  "mcpServers": {
    "fabric-ux-community": {
      "command": "<command from the reviewed community repository>",
      "args": []
    }
  }
}
  
```

Apply the same four gates to every agent-authored change:

1. **SDK:** the method and type exist.
2. **Manifests:** all package declarations agree.
3. **Build:** the real scripts succeed.
4. **Runtime:** the item opens in Fabric with the intended identity and failure behavior.

Appendix D: Python service reference

Example and location	Dependencies	Demonstrates	Production work still required
auth.py, section 4.2	FastAPI, PyJWT, Entra JWKS	Signature, issuer, version, audience, scope	Multitenant issuer policy, resource authorization, key cache, telemetry
onelake.py, section 6.2	requests, pandas, acquired OneLake token	Read with a resource token	OBO acquisition, streaming, retries, large files
jobs.py, section 6.3	FastAPI background task	Long-running execution shape	Fabric remote-job contract, durable state, cancellation, scale
greengrid_saas.py, section 9.2	FastAPI, Pydantic, auth.py	Entra-protected publisher API	Tenant isolation, rate limits, threat model, operational controls
backend_identity.py, section 11.2	azure-identity	Managed identity for a supported target API	RBAC, resource scopes, environment policy
secrets.py, section 12.2	azure-identity, azure-keyvault-secrets	Server-side Key Vault access	Rotation, alerting, break-glass process
telemetry.py, section 12.2	azure-monitor-opentelemetry	Correlated publisher telemetry	Sampling, retention, privacy, customer-facing monitoring
verify_version.py, section 13.1	Python standard library, release version env	Release tag and manifest concordance	Immutable artifacts and component compatibility record

Example and location	Dependencies	Demonstrates	Production work still required
migrate.py, section 17.1	Definition decoder/encoder	Definition schema migration	Backups, idempotency, failure recovery, compatibility tests
Marketplace webhook, Appendix D	FastAPI, Marketplace token validation and fulfillment client	Subscription-event dispatch	Fulfillment v2, retries, idempotency, 24/7 operations

Marketplace webhook example

This skeleton shows event dispatch only. A production implementation must validate the Marketplace token and implement fulfillment v2, retries, idempotency, entitlement state, and continuous operations.

```
# marketplace_webhook.py - react to Microsoft Marketplace SaaS subscription events.
from fastapi import FastAPI, Header, HTTPException, Request

app = FastAPI()

@app.post("/marketplace/webhook")
async def webhook(
    request: Request,
    authorization: str = Header(default=""),
) -> dict:
    if not validate_marketplace_token(authorization):
        raise HTTPException(401, "Invalid Marketplace token")
    event = await request.json()
    action = event.get("action")
    sub_id = event.get("subscriptionId")
    if action == "Subscribe":
        grant_access(sub_id, event["planId"])
    elif action in ("Unsubscribe", "Suspend"):
        revoke_access(sub_id)
    elif action == "ChangePlan":
        update_plan(sub_id, event["planId"])
    return {"status": "accepted"}
```

Appendix E: Release and compliance checklist, diagnostics quick reference

Release sign-off:

Control	Responsible	Expected evidence	Status
Distribution name and publishing tenant match the intended path	Product owner / publisher	Naming decision and tenant record	[]
Frontend/backend use HTTPS under the verified domain	Platform engineering	DNS, certificate, and endpoint evidence	[]
Fabric and Power BI portal framing works	Frontend / QA	Test results from both portal families	[]
Delegation, audiences, OBO, backend credentials, and managed identities are separated	Identity / security	Identity-flow diagram and permission matrix	[]
Definition parts are small, readable, and free of secrets/customer data	Item owner	Definition sample and schema review	[]
Publisher data transfers have residency, retention, isolation, and subprocessor evidence	Security / privacy	Data-flow inventory, privacy material, attestation	[]
Package limits and support/privacy/terms/certification/license links pass	Release owner	Package report and link check	[]
Package version matches the release source and local XML/XSD checks pass	Release engineering	Version check, logs, artifact hash	[]
Item behavior passes in Fabric	Engineering / QA	Create, open, save, error, and migration test record	[]
Target stage passes the separate publishing validator	Publisher	Validator report and finding disposition	[]
Activation, assignment, consent, monitoring, support, and recovery are exercised	Operations / publisher	Runbooks and exercise records	[]

Diagnostics quick reference:

Symptom	Plausible causes	First discriminating test	Expected observation	Next step
Workload does not appear	Tenant setting, capacity, assignment, registration, Dev Gateway	Check published/assigned state separately from local registration	One missing state identifies the administration boundary	Correct that state, then reload the discovery surface
Blank iframe	Local Network Access, DNS/TLS, CSP, route, asset load, JavaScript failure	Inspect iframe request and browser console	Failed network request or first runtime error identifies the next boundary	Fix network/policy first, then route or code
401 from publisher API	Missing bearer token, signature, issuer, version, audience, scope, resource authorization	Log the failed validation category without token contents	Authentication failure is separated from a later authorization denial	Correct token contract or resource policy
401 from Fabric or OneLake	Token requested for the wrong resource, missing scope, failed OBO	Compare token audience/scopes with the called endpoint	Audience mismatch or downstream denial becomes visible	Request the correct resource token or fix permissions
Item shows no data	Valid empty response, failed request, stale component state, wrong source item/path	Inspect HTTP status/body and UI state transition	Distinguishes empty data from transport or rendering failure	Follow data, authorization, or UI branch
Job never starts	Missing scheduler declaration, endpoint resolution, remote authentication, queue failure	Check whether Fabric called the publisher endpoint	Callback absent points to manifest or endpoint. Callback present points to service	Continue at the identified side
Wrong version appears	Local precedence, inactive version, different assignment, cache	Disconnect Dev Gateway and inspect active version/assignment	Version source becomes identifiable	Correct activation/assignment or local session
Publishing review fails	Validator finding, missing evidence, broken support link, offer/publisher mismatch	Map each finding to its requirement and evidence owner	Each finding has a reproducible rule or an explicit open question	Fix, document, or escalate the specific rule

Appendix F: Glossary and resources

Reference record

Field	Recorded value
Toolkit repository	microsoft/fabric-extensibility-toolkit
Pinned commit	dacab1b391d03010ba61a0446126d05515c487f0
Latest observed tag	v2026.03 at commit fbdc891e83d14fbfd4f7e7e27194fd97f153ed , five commits behind the implementation pin
Frontend SDK declaration	@ms-fabric/workload-client range ^3.1.1. No lockfile proves the resolved version
Publishing validator	Tag v2025.12.1, commit 78e17c385ad182f4a293dcab02c2eaa50aa7b361 . Source-inspected, not run
Technical verification	September 17, 2026
Editorial revision	September 17, 2026
Scope checked	Primary documentation, repository structure and types, Markdown structure, static snippet checks, Mermaid rendering, HTML generation, and PDF generation
Scope not executed	Fabric tenant setup, remote endpoint calls, package upload, publishing validator, consent or assignment, and Marketplace fulfillment

Provenance register

Rule	Scope	Exact source	Version or date	Status	Documentary decision
New projects use Extensibility Toolkit terminology	Product naming	Toolkit overview , pinned README	Commit dacab1b... , checked Sep. 17, 2026	Established	Use Extensibility Toolkit. Refer to WDK only for legacy or still-current pages
acquireFrontendAccessToken returns an object whose bearer is in token	Frontend SDK examples	Declared interface , toolkit return type , toolkit usage	Declared range ^3.1.1, no resolved lock	Established	Use tokenResult.token. Do not copy stale acquireAccessToken examples

Rule	Scope	Exact source	Version or date	Status	Documentary decision
v2 custom-API aud is the API client ID. v1 rules differ	Publisher API validation	Entra access-token claims	Checked Sep. 17, 2026	Established, scenario-dependent	Keep the teaching validator single-tenant and v2-specific. State the v1 distinction
Fabric-initiated remote calls use SubjectAndAppToken1.0	Jobs and lifecycle endpoints	Remote authentication	Checked Sep. 17, 2026	Established	Do not apply this contract to browser calls into a publisher API
Remote hosting exists in the tagged toolkit, while general publishing text remains narrower	Hosting and publication	v2026.03 release notes, general requirements	Tag v2026.03, checked Sep. 17, 2026	First-party inconsistency	Describe the current architecture and require target-stage confirmation before a publication promise
The package contains manifest material under BE and FE, not the hosted application binaries	Package output	Manifest overview	Checked Sep. 17, 2026	Established	Show source and built-package trees separately
An item definition is control-plane state, separate from business data	Item persistence and ALM	Store an item definition	Checked Sep. 17, 2026	Established	Keep compact text parts in the definition and place business data in OneLake or an explicit publisher store
Current setup uses SetupWorkload.ps1. Setup.ps1 is a compatibility wrapper	Local setup	Setup wrapper, implementation signature, invalid final handoff	Commit dacab1b...	Source contract established, execution status qualified	Record the working directory and parameters. Do not call the pinned path execution-verified
Selected tenants, Preview, and GA are distinct distribution stages	Distribution and naming	Publishing overview	Checked Sep. 17, 2026	Current platform behavior	Keep actor, prerequisite, validation, and result separate for each stage
Public pages give three different trial answers	Public publication	Optional for both stages, required for both, optional then required	Checked Sep. 17, 2026	First-party inconsistency	Do not invent a universal rule. Confirm against the active validator and publishing team
Publishing self-validation uses the separate validator repository	Publication evidence	Validator README, CLI options	Tag v2025.12.1	Source contract established, execution not performed	Use canonical Preview or GeneralAvailability and retain the generated evidence. Do not present self-validation as Microsoft approval
Public Admin Workloads APIs list and assign but do not document package upload	Release automation	Admin Workloads API	Checked Sep. 17, 2026	Current public API surface	Keep package upload as a documented Admin Portal step

Documented first-party contradictions

Conflict	Exact evidence	Practical decision
The authentication page defines acquireFrontendAccessToken and AccessToken.token, then later uses stale acquireAccessToken and expiresOn examples	Declared API, stale call, undeclared expiry	Follow the declared interface and pinned toolkit usage: <code>acquireFrontendAccessToken(...).token</code>
Entra's version-specific claims reference says v2 aud is the client ID, while a broader article says APIs must accept an App ID URI	Claims reference, broader sentence	Use the version-specific rule: v2 client ID, v1 client ID or resource URI
One remote-auth sample reads ms-client-tenant-id, while lifecycle docs, REST contracts, and toolkit code use x-ms-client-tenant-id	Remote sample, lifecycle contract, toolkit code	Use <code>x-ms-client-tenant-id</code>
Remote guidance allows operations without a subject token, while pinned job handlers use authentication that requires one by default	Absence cases, pinned default	Define explicit app-only authorization for operations that legitimately lack user context
Official repositories expose different job route shapes	Toolkit routes, Python sample create route	Treat the exact job URL contract as unresolved and confirm it for the target stage
Setup guidance points to a wrapper, the pinned repository prefers the implementation, and that implementation calls an unsupported build parameter	Learn setup, wrapper, invalid handoff	Mark the commands source-inspected and recheck upstream before execution
Validator README says Node.js 14 or later, while its locked Commander dependency requires Node.js 20 or later	README prerequisite, locked engine	Use Node.js 20 or later for the pinned validator
Fabric publishing text overstates verified publisher status compared with the owning Entra definition	Fabric statement, Entra definition	Treat publisher verification as organizational identity, not security or compliance certification

Conflict	Exact evidence	Practical decision
Legacy and split Preview/GA requirement matrices disagree on ALM, Private Links, Data Hub, labels, and Monitoring Hub	Legacy matrix , split workload matrix , split item matrix	Do not synthesize a universal matrix. Confirm the active validator and publishing-team rule for the target stage

Glossary

Term	Definition and relationship
Workload	Publisher product registered with Fabric, containing one or more item types and publisher-hosted experiences
Item type	Release-time contract declared by paired platform and frontend manifests
Item instance	Workspace object created by a user from an item type
Manifest package	Versioned .nupkg containing workload, product, item-type declarations, assets, and locale material
Item definition	Small control-plane parts describing one item instance, separate from business data
Data plane	OneLake or another explicitly governed store holding files, tables, and large results
Delegated token	Resource-specific token representing the signed-in user
OBO	Backend exchange of a subject token for a token targeting another resource
Application token	Token representing a backend application or managed identity rather than a user
Fabric.Extend	Permission required for the workload integration flow
Workload Hub	Fabric discovery, consent, installation, and assignment surface
Microsoft Marketplace	SaaS listing, licensing, and commercial surface used by public publishing
ActivityId / RequestId	Correlation identifiers available on documented Fabric paths and propagated by the publisher where present

Primary references

Toolkit and item model:

- [Extensibility Toolkit overview](#)
- [Architecture](#)
- [Manifest package overview and limits](#)
- [Store an item definition](#)
- [Official toolkit repository](#)

Development and identity:

- [Setup guide](#)
- [Frontend authentication API](#)
- [Remote authentication](#)
- [Entra access-token claims](#)

Publishing and administration:

- [Publishing overview](#)
- [Workload publishing requirements](#)
- [Publish a workload to a tenant](#)
- [Publishing validator](#)
- [Official validator repository](#)
- [Fabric Admin Workloads APIs](#)
- [Entra publisher verification](#)

Commerce:

- [Microsoft Marketplace SaaS offer options](#)
- [Microsoft co-sell requirements](#)

Maintenance rule

- Revalidate the toolkit commit/tag, SDK package, manifest schemas, setup/build scripts, tenant settings, publishing requirements, validator, and Marketplace rules whenever any of those dependencies changes.
- Record the **editorial modification date** separately from the **technical verification date**.
- State whether a claim was checked through documentation, source inspection, compilation, rendering, a local test, a Fabric tenant test, or an official publishing review.
- Treat an unresolved first-party discrepancy as a localized open question with a practical constraint, not as permission to choose the more convenient rule.

Appendix G: Quick reference

A scannable summary of the manuscript.

Core commands (details and caveats are in Appendix B):

```
pwsh ./scripts/Setup/SetupWorkload.ps1 -WorkloadName "Org.YourWorkload"
pwsh ./scripts/Setup/CreateNewItem.ps1 -ItemName "Forecast" -srcItemName "HelloWorld"
pwsh ./scripts/Run/StartDevServer.ps1
pwsh ./scripts/Run/StartDevGateway.ps1
pwsh ./scripts/Build/BuildManifestPackage.ps1 `
  -Environment prod `
  -ValidateFiles $true
```

Verify the .nupkg file, archive contents, version, and hash.

The manifest package:

File	Declares
WorkloadManifest.xml	workload identity, hosting mode, Entra applications, endpoints
Product.json	product presentation, support, privacy, terms, and listing metadata
{Item}.xml	platform definition for one item type
{Item}.json	frontend behavior and presentation for the same item type

The identity split. Use a delegated frontend token for its requested resource. Use backend OBO when the service must exchange the subject token for another resource. Use a backend application credential for the documented remote flows that require it, stored and rotated server-side. Use managed identity for supported publisher service calls.

Developer mode to production, the swaps:

Development	Production
Dev Gateway from localhost	frontend hosted under a verified domain
developer-mode dev instance	.nupkg uploaded, activated, and assigned
development app configuration	production frontend and backend app registrations
local developer identity	delegated user, backend application, and managed-identity flows
local manifest build	package build with XML/XSD checks plus publishing validation

Diagnostics use observations, not symptom shortcuts. For a 401, separate authentication, authorization, token audience, and transport. For a blank iframe, inspect network loading and the browser console before selecting a boundary. Propagate ActivityId and RequestId only on paths where they are present. Use the full table in Appendix E.

The naming fork. Org. [Name] is for the publishing tenant. [Publisher].[Workload] supports selected tenants, Preview, and GA. Public stages require the Fabric publishing review and a Microsoft Marketplace SaaS offer.

The four movements. Understand the business and technical model, develop it by hand and with repository-guided assistance, take it to production, then distribute it through tenant assignment, Workload Hub, and Microsoft Marketplace.